Training–Rollout Mismatch in Low-Precision LLM Reinforcement Learning: A Survey

Zunhai Su
The University of Hong Kong

Abstract: Quantized rollouts reduce the cost of language-model reinforcement learning, but they can also change the distribution of training experience and the updates computed from it. This survey connects these effects through a common formulation of learner, behavior, proximal, and reference policies. We synthesize twenty-nine studies around four component-level interventions: aligning computation, preserving numerical fidelity and parameter-update visibility, correcting gradient estimation, and controlling policy updates. Optimization schedules provide a complementary perspective. The resulting framework separates a method's operative mechanism from its precision, architecture, and collection setting. We examine how rounding differences, conditional score drift, and signed update selection affect learning, and compare the evidence available for representative combinations. Across these studies, smaller probability gaps, stable reward curves, and faster generation capture different outcomes; none alone establishes more efficient learning. We relate the reported results to their baselines, evaluated horizons, information requirements, and end-to-end costs, and identify open questions in harmful-mismatch diagnosis, component composition, and stateful collection.

Keywords: quantized rollout; low-precision reinforcement learning; training–inference mismatch; importance sampling; quantization-aware training.

Literature snapshot: October 20, 2026.

Survey resources: https://github.com/ZunhaiSu/awesome-low-precision-rl

Living survey, under active development. Suggestions, corrections, and research collaborations are welcome via GitHub Issues.

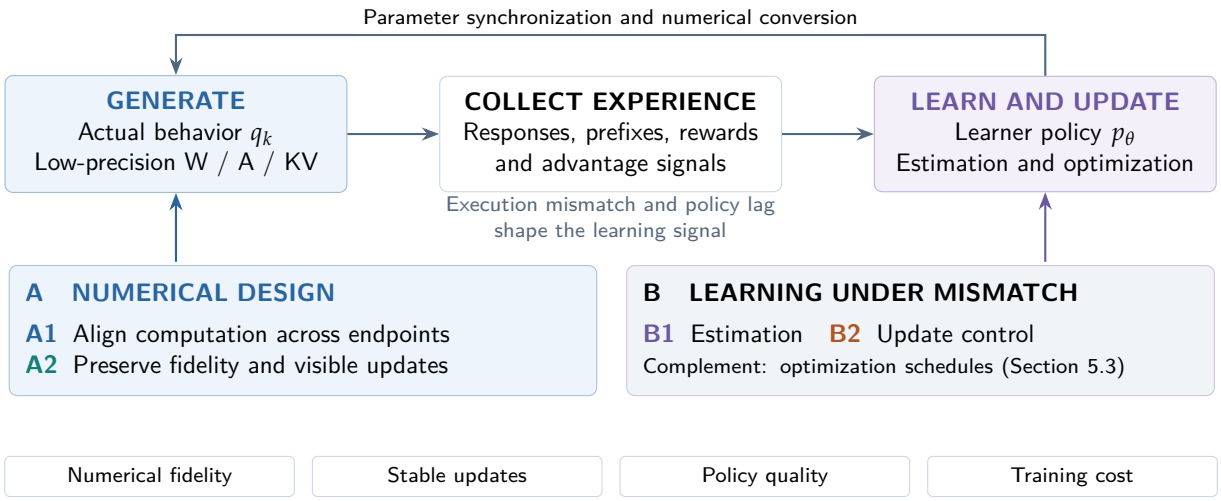


Figure 1: Survey at a glance. Numerical methods act on learner or sampler execution; learning methods change how samples drive updates. Their effects return through the rollout–learning loop. W, A, and KV denote weights, activations, and key–value cache.

Contents

1 Introduction and Scope **4**

1.1 Why Low-Precision Rollouts Complicate Policy Optimization 4

1.2 Scope and Related Work 4

1.3 Perspective and Contributions 5

2 Background and Problem Formulation **6**

2.1 Learner, Behavior, Proximal, and Reference Policies 6

2.2 Execution Mismatch versus Policy Staleness 7

2.3 A Component-Level Taxonomy of Interventions 8

2.4 Numerical Regimes and Contribution Roles 10

3 How Execution Mismatch Affects Learning **10**

3.1 Sources and Propagation of Execution Discrepancy 10

3.2 Distorted Estimation and Update Decisions 10

3.3 Feedback across Optimization Steps 12

3.4 Diagnosing the Effects of Mismatch 12

4 Numerical Design and Execution Alignment **13**

4.1 Aligning Learner and Rollout Computation 14

4.2 Controlling Quantization Error and Preserving Parameter Updates 15

4.3 Residual Mismatch and Limits of Alignment 16

5 Learning under Residual Mismatch **17**

5.1 Correcting Gradient Estimation 17

5.2 Controlling the Influence of Policy Updates 19

5.3 Adapting the Optimization Schedule 20

5.4 Compatibility, Bias, and Failure Modes 21

6 Evidence Synthesis and System Trade-offs **21**

6.1 Experimental Regimes . . . 22

6.2 Findings on Stability and Learning Quality . . . 22

6.3 Efficiency Gains and Implementation Costs . . . 27

6.4 Evidence Limits and Coverage Gaps . . . 28

7 Open Problems **29**

7.1 Predicting Harmful Mismatch . . . 29

7.2 Composing Numerical and Learning Interventions . . . 29

7.3 Precision, Staleness, and Long-Horizon Interaction . . . 29

7.4 Generalization and Comparable Evidence . . . 30

8 Conclusion **30**

A Surveyed Papers and Section Mapping **31**

B Source Selection and Version Handling **33**

C Evidence, Assumptions, and Transfer **33**

D Terminology **34**

Reading pathways

Foundations	Methods	Evidence
Policy roles, execution mismatch, and its effects on learning.	Numerical interfaces, estimator correction, and update control.	Within-study findings, system costs, and open problems.
Sections 2–3	Sections 4–5	Sections 6–7

Appendix A: surveyed papers · Appendices B–C: sources and assumptions · Appendix D: terminology

1. Introduction and Scope

1.1. Why Low-Precision Rollouts Complicate Policy Optimization

In language-model reinforcement learning, generation determines the responses, rewards, and histories from which the next policy is learned. Quantizing the rollout engine changes this source of training data as well as its computational cost. A local numerical difference can alter a sampled token, the learning signal assigned to a response, and the parameter update that shapes later generation. The resulting training–rollout mismatch is a low-precision instance of training–inference mismatch (TIM), with consequences that extend beyond the accuracy of a single forward pass.

The practical objective is to learn a capable policy within a finite computational budget. Faster generation contributes to that objective through the quality of the data it supplies and the updates those data support. Numerical agreement, optimization stability, task performance, and runtime capture different parts of this relationship. Aggressive clipping, for example, can stabilize training while discarding useful signal; a more faithful sampler can preserve that signal while reducing the throughput gained from quantization. Their value depends on how efficiently the complete learning process improves the policy.

Existing methods intervene at different points in this process. FlashRL uses actual rollout probabilities to correct learning from a different sampling policy; Jet-RL coordinates learner and sampler precision flow; subsequent studies act on quantization, clipping, gradient estimation, and local update dynamics [1, 3]. These interventions often coexist within one system. Numerical components act on learner or sampler computation, including the interface between them; learning-side rules determine how the resulting samples influence the next policy. Component-level classification preserves these distinct roles within integrated systems.

Figure 2 organizes these components by the computation or update they change. Cross-category assignments preserve the distinct contributions of integrated methods; Appendix A summarizes the corresponding papers.

1.2. Scope and Related Work

This survey focuses on numerical and execution mismatch in LLM reinforcement learning, especially quantized rollout and related low-precision training configurations. It covers a higher-precision learner paired with a lower-precision sampler, as well as low precision on both sides. Weight, activation, and key–value (KV) cache quantization affect different computational interfaces. Policy staleness enters where parameter changes interact with those interfaces.

The twenty-nine focal contributions comprise nineteen method or system studies, nine supporting formulation, theory, or general mismatch studies, and one exploration boundary case. These roles describe their use in the synthesis: MiniRL, for example, develops the formulation and also evaluates FP8 rollout [14]. The literature snapshot is October 20, 2026. Appendix B describes source selection and version handling.

Broader surveys address low-precision LLM training, rollout strategies, and reinforcement learning for reasoning models [33–35]. The distinctive concern here is the connection between actual sampling probabilities, numerical execution, and the update computed from sampled data. General post-training quantization, pretraining-only methods, and speculative-draft acceleration enter only where they clarify that connection.

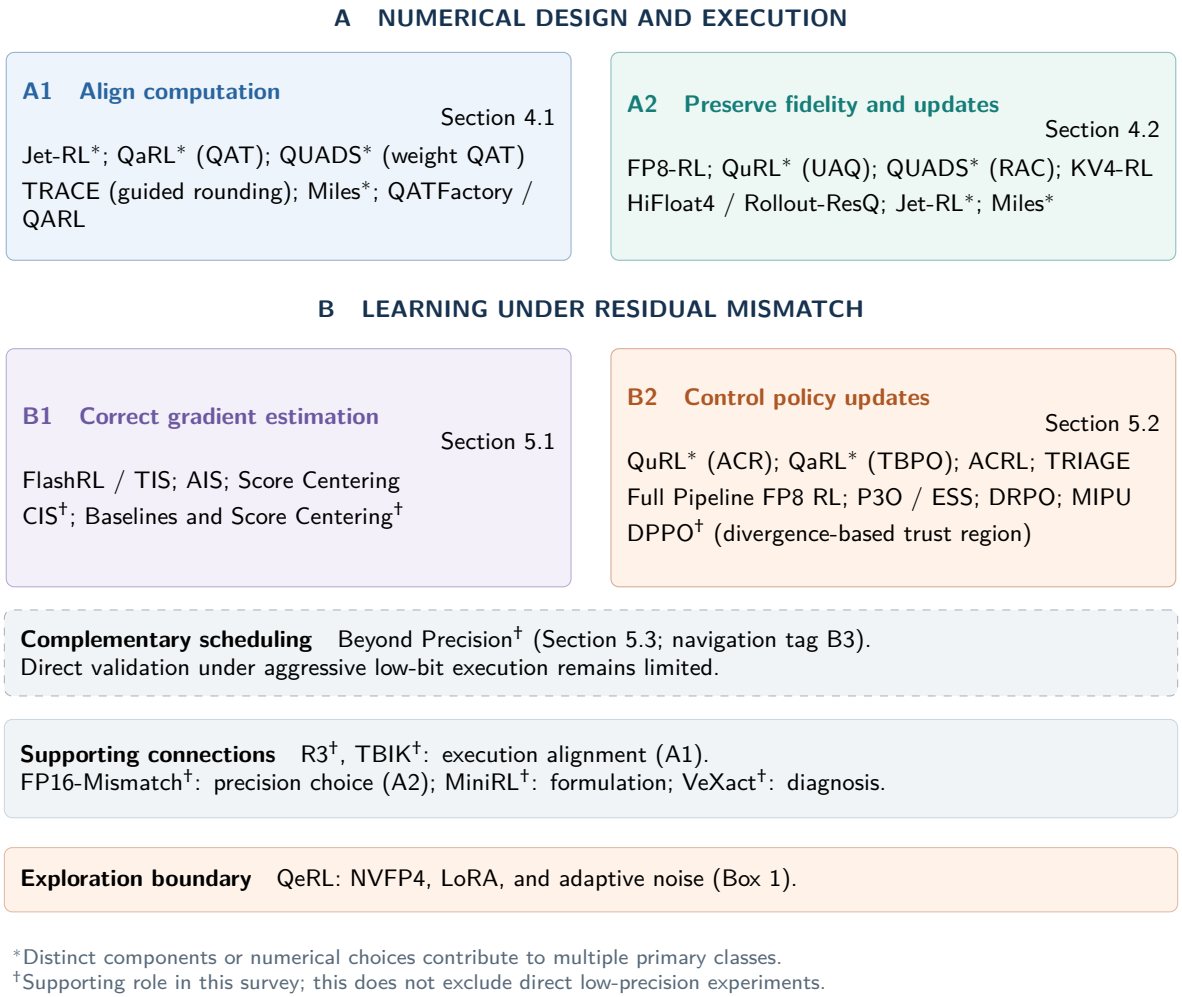


Figure 2: Component-level taxonomy of all twenty-nine focal studies. The four primary classes describe direct interventions; repeated names identify distinct components. Scheduling, supporting connections, and the QeRL exploration boundary are shown separately. Numerical regimes remain independent comparison axes. Appendix A gives the complete section mapping.

The collection supports a focused qualitative synthesis of primary studies. Numerical format, architecture, adaptation, and collection protocol remain separate comparison axes. Table 6 summarizes the represented regimes; Appendix D defines recurring abbreviations.

1.3. Perspective and Contributions

Numerical fidelity and learning correction are coupled: the sampler determines which experience is collected, and the update rule determines how that experience changes the next policy. We develop three contributions around this connection. A common policy formulation fixes the meaning of probability comparisons. A component-level taxonomy explains how numerical and learning interventions differ and combine. Configuration-specific evidence then separates demonstrated benefits from plausible transfer, linking the mechanisms to policy quality and total cost.

We develop this perspective by connecting policy definitions, failure mechanisms, and method components. The formulation in Section 2 identifies which distribution each probability ratio refers

to; Section 3 traces discrepancies through estimation and optimization. This foundation supports the comparison of numerical and learning interventions in Sections 4–5. Their shared dependencies explain the published findings and practical trade-offs synthesized in Section 6, from which the open problems in Section 7 follow.

2. Background and Problem Formulation

2.1. Learner, Behavior, Proximal, and Reference Policies

Let x be a prompt, $y_{1:T}$ a response, and $h_t = (x, y_{<t})$ its prefix. A policy is specified by both a parameter state and the computation used to evaluate it. Write $p_\theta(a \mid h)$ for the current learner policy and $p_k(a \mid h)$ for the same learner-side path at sampling snapshot θ_k.

The behavior policy $q_k(a \mid h)$ is the distribution that actually sampled the token. It includes the rollout engine, numerical representation, and sampling transformation. Temperature and vocabulary truncation therefore belong to its definition: a raw-model log-probability returned by an engine need not equal the probability after decoding transformations.

The collection protocol is part of the behavior law. Under deterministic execution and version schedules, the generated history determines the sampler state, including cache entries computed under earlier parameter versions. The subscript k abbreviates that rollout path. Cache provenance identifies the parameter versions and numerical paths that produced retained cache entries; a checkpoint identifier alone cannot specify this state.

A proximal policy p_{prox} supplies the denominator of a policy-update constraint. A reference policy p_{ref} supplies a regularization target. These are different roles: a frozen pre-update learner may be proximal, while a much earlier policy is the KL reference. Neither is automatically the policy that generated every token. PPO and GRPO provide familiar contexts for these distinctions [30, 31]; mismatch-aware formulations require them to remain explicit [1, 14].

Outcome-based GRPO provides a concrete running example. For each prompt, the sampler generates a group of responses and a task-dependent scorer assigns each response a reward R_i. A rule-based verifier can supply this score; a learned reward model is not required. A common group-relative advantage is $A_i = (R_i - \bar{R})/(\sigma_R + \epsilon_A)$, although practical variants can omit standard-deviation normalization. The advantage is held fixed during the policy update and is typically shared by the response's valid tokens. It determines which sampled actions are encouraged or discouraged, while probability ratios connect that signal to the learner parameters [31]. The KL reference, when used, has a separate regularization role.

Policy identity also determines the object whose improvement matters after synchronization. MIPU explicitly distinguishes improving the learner from improving the resulting inference policy [23]. This distinction becomes consequential when conversion and rollout execution change the distribution realized by an accepted update.

Table 1: Policies and quantities used throughout the survey. Execution paths are part of policy identity.

Symbol	Meaning
p_θ	Current learner policy under its specified execution path.
p_k	Learner-side policy evaluated at sampling snapshot θ_k.
q_k	Actual behavior policy that sampled the token.
p_{prox}	Policy used to define the proximal update ratio.
p_{ref}	Policy used for KL regularization; not the behavior denominator.
Δ_k	Same-snapshot log-probability difference, $\log p_k - \log q_k$.
d_θ	Current learner–behavior gap, $\log p_\theta - \log q_k$.
A_t	Advantage or other policy-gradient learning signal.
$s_\theta(a,h)$	Score vector, $\nabla_\theta \log p_\theta(a \mid h)$.
$c_t, \bar{c}_t$	Behavior-to-proximal correction and its truncated or masked form.
$r_t(\theta)$	Proximal update ratio, $p_\theta / p_{\text{prox}}$.
$W_\theta(y \mid x)$	Complete-response likelihood ratio, P_θ / Q.
N, sg	Valid-token count and stop-gradient operator in Equation (9).

2.2. Execution Mismatch versus Policy Staleness

At a fixed prefix and token with positive relevant probabilities,

$$\frac{p_\theta(a \mid h)}{q_k(a \mid h)} = \underbrace{\left(\frac{p_\theta(a \mid h)}{p_k(a \mid h)}\right)}_{\text{parameter change / lag}} \times \underbrace{\left(\frac{p_k(a \mid h)}{q_k(a \mid h)}\right)}_{\substack{\text{same-snapshot} \\ \text{execution / decoding}}}. \tag{1}$$

The first factor captures parameter change since collection, including asynchronous version lag; the factorization requires no independence assumption. The second compares execution at the sampling snapshot. It isolates numerical differences when decoding is matched and also includes decoding differences when temperature or top-p filtering changes. The intermediate policy p_k is a conceptual reference and need not be retained by the implementation. In partial rollout, a response can span collection boundaries, so $k = k(t)$ may vary across tokens. If caches survive synchronization, their earlier computations remain part of the behavior path, so reconstructing q_k requires cache provenance or the realized sampling probabilities as well as the current checkpoint.

Figure 3 separates these snapshot comparisons from the two policy roles used by the loss. A proximal anchor and a KL reference can be chosen independently of the actual sampler.

Two limiting cases make the distinction concrete. At the sampling snapshot, the learner-side parameter-change factor is one while execution mismatch can remain. With matched execution and decoding, the second factor can instead be one while policy lag remains. A numerical intervention and a staleness correction therefore solve different parts of the same ratio.

Probability ratios also require support conditions. A token observed from q_k has a defined denominator, but that alone does not establish that q_k covers every action with positive target probability. Truncation sampling can violate this condition. Similarly, a token-level change of measure is not automatically a correction for the prefix distribution. At fixed h,

$$\mathbb{E}_{a \sim q_k(\cdot \mid h)}\left[\frac{p_\theta(a \mid h)}{q_k(a \mid h)} f(a,h)\right] = \mathbb{E}_{a \sim p_\theta(\cdot \mid h)}[f(a,h)] \tag{2}$$

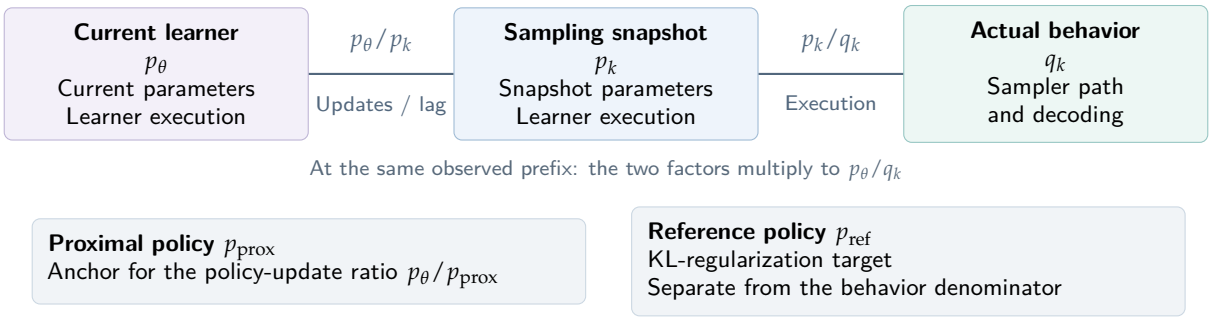


Figure 3: Policy identities and loss roles. Horizontal links denote the ratio comparisons in Equation (1), rather than data flow. Execution differences isolate numerical mismatch when decoding is matched. The lower cards identify separate roles in optimization.

holds under support and integrability assumptions. It says nothing by itself about whether the outer distribution over h is the desired one. It also does not undo changes to group-relative advantages induced by collecting different responses.

For comparison, the full-response likelihood ratio is

$$W_\theta(y \mid x) = \frac{P_\theta(y \mid x)}{Q(y \mid x)} = \prod_{t=1}^{T} \frac{p_\theta(y_t \mid h_t)}{q_{k(t)}(y_t \mid h_t)}. \tag{3}$$

Here P_θ and Q are the induced response laws, and the denominator factors must be their actual conditional probabilities under the documented protocol. Unobserved execution randomness requires marginalizing that state or extending the likelihood to the corresponding execution events. The two response laws use a common stopping convention; an end-of-sequence decision is included when applicable. For a fixed behavior law and a parameter-independent reward R, the usual likelihood-ratio identity gives

$$\nabla_\theta \mathbb{E}_{P_\theta}[R] = \mathbb{E}_Q\left[W_\theta R \sum_{t=1}^{T} s_\theta(y_t, h_t)\right] \tag{4}$$

under support, integrability, and differentiation assumptions. The product illustrates why exact response-level correction can have high variance; multiplying only the current token ratio into an arbitrary sampled advantage is a different estimator. Nor does Equation (4) directly justify reusing group-normalized advantages computed under Q. Score Centering explicitly distinguishes sequence-level importance sampling from practical token-level corrections [10].

A two-step example in Figure 4 makes the distinction tangible. The second-token conditionals agree exactly, yet the first-token histories have different probabilities. A correction using only the second-token ratio therefore leaves that prefix imbalance, whereas the full-response ratio includes the earlier decision.

2.3. A Component-Level Taxonomy of Interventions

We classify operative components by the object they directly change. **A: numerical interventions** act on representation or execution: A1 aligns learner and sampler computation, while A2 preserves numerical fidelity or the visibility of parameter updates. **B: learning interventions** act on the conversion of experience into an update: B1 corrects gradient estimation, while B2 regulates update

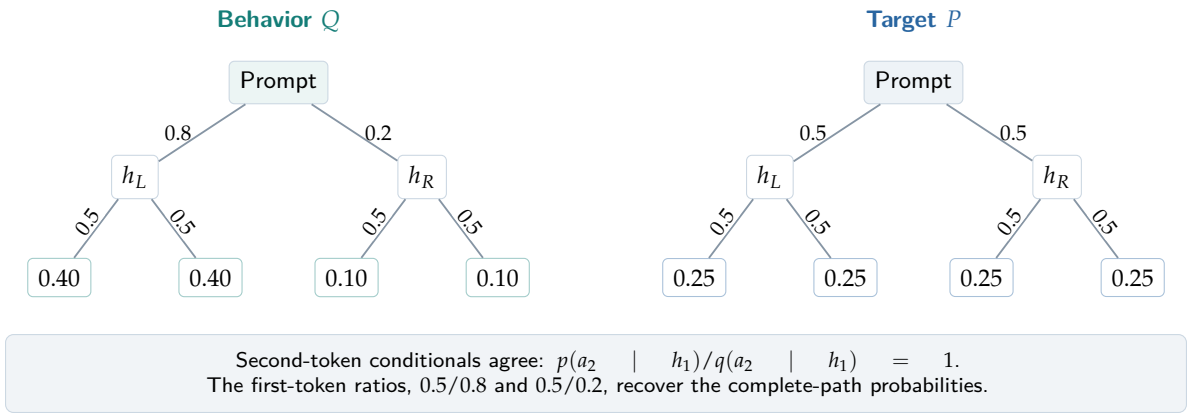


Figure 4: Conditional and response-level correction in a two-step probability tree. Leaf values are complete-response probabilities. Equal second-token conditionals coexist with different prefix masses; the product ratio restores the four target probabilities of 0.25. This is a constructed illustration, not an experimental result.

influence or acceptance. These four primary classes organize Sections 4–5. Scheduling is retained as a complementary topic in Section 5.3, with B3 used only as a navigation tag for that discussion.

Classification follows the operation and its target, while secondary effects can cross class boundaries. Alignment may improve fidelity, and both estimator correction and update control can use multiplicative weights. B1 targets a distributional expectation; B2 regulates sample influence on the update. QaRL and QuRL therefore occupy several classes through coequal components. An adopted correction remains separate from a system's proposed contribution; discussion locations indicate expository placement. Scheduling does not receive equal weight in the primary taxonomy because its direct low-bit evidence is thinner.

Table 2: Four primary intervention classes and a complementary scheduling topic. Assignments concern operative components; a paper may contribute to several classes.

Class	Direct object	Representative components	Residual limitation
A1	Shared computation	Precision flow; aligned QAT; guided rounding	Unmatched execution interfaces.
A2	Quantization fidelity	UAQ; residual compensation; calibration; selective precision	Resolution of useful parameter changes.
B1	Gradient estimator	TIS; AIS; CIS; Score Centering	Target expectation and approximation bias.
B2	Update influence	ACR; TBPO; calibrated clipping; direction-aware controls	Retained signed learning signal.
B3	Complement: schedule	Mismatch-informed LR adaptation	Direct low-bit validation remains limited.

2.4. Numerical Regimes and Contribution Roles

The same intervention can operate under different W/A/KV precisions, dense or mixture-of-experts (MoE) architectures, low-rank or full-parameter updates, and synchronous or partial collection. Native kernels and quantize–dequantize simulation also implement different numerical paths. In particular, "FP4 training" leaves open the precision of master weights, backward computation, and optimizer states.

These configuration axes also preserve attribution: integrating a correction into a quantized system is distinct from introducing that correction. A study's contribution role and the regimes tested in its experiments are recorded separately.

3. How Execution Mismatch Affects Learning

3.1. Sources and Propagation of Execution Discrepancy

Scale selection, rounding, accumulation, kernel choice, batch shape, and cached state can all alter a numerical policy's output distribution. Two paths using the same nominal format can consequently disagree. FP16-based analysis illustrates the role of precision selection, while R3 exposes the effect of discrete MoE routing decisions [15, 16]. Routing disagreement can also arise independently of quantization.

Cross-TP deterministic inference isolates another execution source: changing the reduction order can change probabilities even when parameters and nominal precision match. TBIK coordinates reduction trees across tensor-parallel sizes, providing a matched-execution reference rather than a low-bit accelerator [28]. At more aggressive precision, HiFloat4 studies outlier-driven underflow in rollout activations; its precision ablations connect this local information loss to degraded RL outcomes [24].

Shared-prefix comparison holds the token history fixed and isolates differences between conditional distributions. Free-running comparison allows different samples to change the histories visited later. These settings expose different stages of propagation: a small conditional error can redirect generation, while a favorable shared-prefix average can conceal disagreement on histories reached by only one policy.

Discretization introduces a further asymmetry. An update can be visible in the learner's parameter space but fail to change the sampler's quantized representation. Conversely, a small change near a rounding boundary can abruptly change a codeword. QuRL studies the former through update-aware quantization; TRACE targets inconsistency in rounding outcomes [5, 13]. The relevant object is therefore not only error magnitude, but its relationship to the evolving parameters and the discretization boundaries.

Figure 5 organizes the resulting learning effects into relative influence, expected direction, and update selection. These routes can coexist, which explains why one aggregate disagreement statistic can miss the active mechanism.

3.2. Distorted Estimation and Update Decisions

At an observed prefix, numerical disagreement can change an update through its weight, its expected direction, or the clipping branch it activates. Absolute probability error captures only part of this

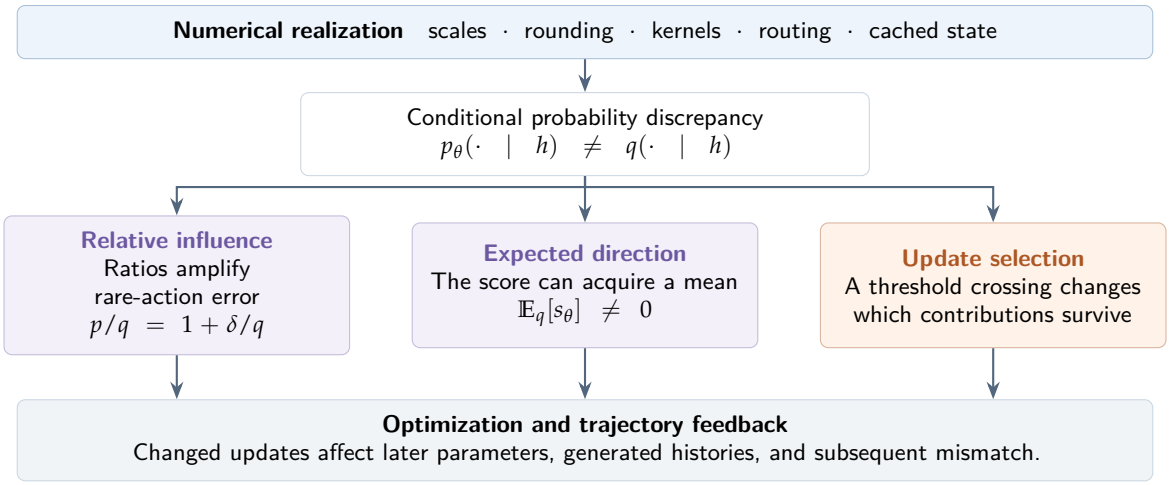


Figure 5: Three routes from execution discrepancy to learning dynamics: relative influence, expected score direction, and update selection. These effects can coexist and feed back through later updates and generated histories. Arrows indicate possible propagation under the policy and estimator assumptions in the text.

effect. Writing $p = q + e$ gives $p/q = 1 + e/q$: the same absolute perturbation has greater relative influence when q is small. Tails and confidence-conditioned statistics therefore reveal structure that a mean absolute error can hide. CIS uses this confidence dependence in its correction design [20].

DPPO makes a related distinction between relative ratios and moved probability mass [29]. A large ratio on a rare token need not represent a large distributional shift, while a moderate ratio on a common token can move substantial mass. This motivates separating the geometry of an update constraint from the numerical size of its sampled ratio.

Estimation can also acquire a systematic drift. For a normalized differentiable learner distribution, $\mathbb{E}_{a\sim p_\theta}[s_\theta(a,h)] = 0$. Under behavior sampling, the corresponding expectation need not vanish. At fixed h,

$$\mathbb{E}_q[As_\theta] = \mathrm{Cov}_q(A, s_\theta) + \mathbb{E}_q[A]\mathbb{E}_q[s_\theta]. \tag{5}$$

The expectations are conditional on the prefix and include the sampled action and any random continuation used to form A. Centering advantages across a prompt group need not center them at each prefix. Score Centering uses this distinction to motivate an additive correction [10]: it removes a first-moment drift term while leaving the behavior distribution itself unchanged.

Mismatch additionally affects decisions made by a surrogate objective. In a clipped policy-gradient objective, changing a ratio can change which branch is active and therefore whether a token contributes gradient. Thus, a perturbation can matter through a threshold crossing even when its absolute magnitude appears modest. The full-pipeline FP8 study isolates a failure involving excessive clipping of negative-advantage tokens [11]. This is a different intervention target from merely decreasing the variance of a sampled importance weight.

The sign dependence can be seen without assuming a particular quantizer. Let $\ell(r,A) = \min\{rA, \mathrm{clip}(r, 1-\epsilon, 1+\epsilon)A\}$, with $0 < \epsilon < 1$ and fixed A. Away from the two boundaries,

$$\frac{\partial \ell}{\partial r} = \begin{cases} A\,\mathbf{1}\{r < 1+\epsilon\}, & A > 0, \\ A\,\mathbf{1}\{r > 1-\epsilon\}, & A < 0, \\ 0, & A = 0. \end{cases} \tag{6}$$

Thus the active clipping side depends on the learning signal. For an illustrative objective that clips the combined learner–behavior ratio, a same-snapshot ratio of 0.7 with $\epsilon = 0.2$ already suppresses a negative-advantage token, even before a learner update. This is not the decoupled surrogate in Section 5.1; the example explains why naming the clipped ratio is essential to interpreting a failure or a repair.

Figure 6 makes this sign dependence explicit for the standard surrogate. It also shows why clipping the combined learner–behavior ratio can suppress an update that a proximal ratio would still permit.

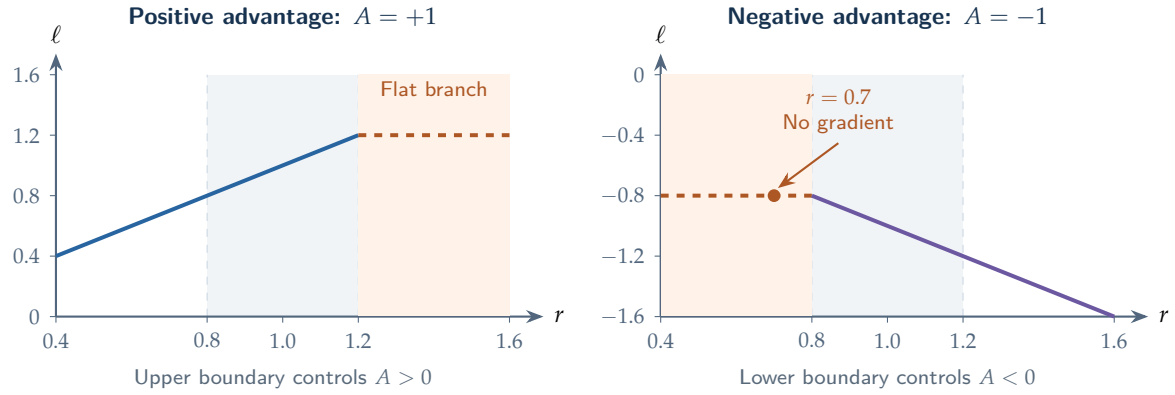


Figure 6: Signed clipping in Equation (6), with $\epsilon = 0.2$ and $A = \pm 1$. The gray interval is $[0.8, 1.2]$; amber segments are flat. At $r = 0.7$, a negative-advantage token contributes no gradient through this objective. The clipped policy ratio determines the interpretation. This is a constructed illustration, not an experimental result.

3.3. Feedback across Optimization Steps

The learning loop turns a local discrepancy into a dynamic process. Updates based on sampled responses change the learner; synchronization transfers those changes to the sampler; subsequent rollouts come from the updated numerical realization. Stability depends on this feedback, not solely on whether one frozen batch has a small error.

The magnitude of a learner–behavior gap alone does not determine how an update changes it. In a simplified local picture, reducing a learner probability that is already below the behavior probability can enlarge their log gap. ACRL and TRIAGE use update-direction information in their controls [9, 12]. For a shared-parameter network, however, one token's update changes other probabilities too. The sign argument supplies local intuition; interactions through shared parameters determine the scope of any stability result.

Optimization scale adds another feedback channel. Beyond Precision studies the relationship between mismatch and optimization dynamics and proposes response-length-triggered learning-rate decay [18]. This connects numerical observations to the scale of optimization, while leaving transfer of the schedule to aggressive FP4 execution open.

3.4. Diagnosing the Effects of Mismatch

Diagnosis begins with the comparison used to isolate an effect. Replacing an inference engine can simultaneously change arithmetic, batching, memory use, and generated trajectories. A resulting

quality difference cannot automatically be assigned to the precision format. VeXact constructs a zero-mismatch reference to help separate execution mismatch from other training effects [19]. It supplies a diagnostic counterfactual against which the effects of accelerated execution can be interpreted.

The diagnostics in Table 3 locate different effects within the chain from execution to learning. A controlled comparison connects a change in conditional probabilities to an estimator or update decision; the training trajectory reveals its accumulated consequence.

Table 3: Mechanisms, observable diagnostics, and intervention targets. Diagnostics characterize different effects and do not by themselves predict downstream quality.

Mechanism	Diagnostic at a specified policy and prefix	Related intervention
Relative influence	Tails of p/q, conditioned on token confidence; correction truncation or masking rate.	B1: TIS, AIS, and CIS control estimator influence [1, 7, 20].
Conditional score drift	Estimated $\mathbb{E}_q[s_\theta]$ and the contribution $\mathbb{E}_q[A]\mathbb{E}_q[s_\theta]$; reconstruction error matters.	B1: Score Centering removes the exact first-moment term; implementations approximate it [10].
Update selection	Active clipping branches and retained contributions, separated by advantage sign; local directional patterns.	B2: clipping calibration and direction-aware control [11, 12].

Shared-prefix measurements isolate conditional effects, whereas free-running trajectories include changes in the histories visited. Signed update diagnostics additionally depend on the surrogate and fixed advantage signals. Their connection to learning requires evidence over the training trajectory.

Box 1. Quantization and exploration

QeRL combines low-precision execution, low-rank adaptation, and adaptive quantization noise, making exploration an explicit part of the training design [2]. It occupies a boundary of this survey: the numerical perturbation changes the search process as well as agreement between computations. Its results establish a role for quantization noise in that setting, with task utility depending on the induced sampling distribution and the learner's adaptation capacity. They do not establish a benefit from arbitrary cross-engine error.

These mechanisms connect numerical design to the learning signal it preserves. Relative weighting, conditional drift, threshold decisions, and trajectory feedback expose different consequences of the same execution change. The numerical and learning interventions that follow address different parts of that chain.

4. Numerical Design and Execution Alignment

Numerical design can act on the learner, the sampler, or the interface between them. Alignment brings their computations closer at a shared interface; fidelity control preserves information that a low-precision representation would otherwise lose. The distinction matters because two approximate paths can agree without either becoming more accurate relative to a higher-precision model.

4.1. Aligning Learner and Rollout Computation

Jet-RL coordinates the precision flow used by learner and sampler [3]. Its unified FP8 design matches corresponding operators and quantization granularity across the endpoints. This makes the computation, rather than the format label, the interface of alignment.

QaRL implements such coupling through rollout-aligned quantization-aware training (QAT): the learner's forward pass reflects the representation used during generation [6]. QUADS takes a deliberately asymmetric approach, fake-quantizing learner weights while retaining higher-precision learner activations [8]. Both designs bring part of the learner computation closer to the sampler, but QUADS leaves activation error to a separate rollout-side component. These designs illustrate selective alignment; uniform precision across the whole model is neither its definition nor a prerequisite. QAT emulation and native low-precision kernels nevertheless remain distinct numerical paths.

TRACE aligns realized rounding decisions [13]. A shared FP4 format and scale can still send nearby learner and rollout activations to different codewords. TRACE records rollout-side quantization information and uses it to guide the learner's corresponding quantization site. With exact guidance and fixed rollout scales, it selects from the learner value's two neighboring codewords the one nearest the rollout codeword. Figure 7 illustrates why this can reduce cross-path discrepancy even when it slightly increases the learner's own quantization error.

The practical mechanism approximates this reference with compact code information. By default, TRACE caches low mantissa bits and scales from the latter half of the layers, pairing records by trajectory, token position, and quantization site. Learner-side values and rollout scales help reconstruct the local reference; the compact record does not identify an arbitrary original codeword exactly. The reported adjacency pattern motivates this compression, rather than supplying a universal recovery guarantee.

Precision on the two endpoints remains distinct. The main rollout setting combines NVFP4 expert weights and activations with NVFP4 KV storage in softmax-attention layers; other modules remain BF16. The learner uses rollout-guided QAT: forward fake quantization and a BF16 backward path. This is not evidence for native-FP4 trainer acceleration. Routing replay and token-level importance correction address other residual differences, while asynchronous collection can still introduce policy lag. The local rounding property therefore does not imply full-network agreement or removal of staleness [13].

System implementations extend alignment across the model lifecycle. Miles coordinates checkpoint conversion, learner forward computation, rollout kernels, and live weight export through shared quantization rules [25]. Its FP8, MXFP8, and NVFP4 recipes retain higher-precision exceptions; MXFP8 and NVFP4 are marked Beta. QATFactory's QARL path pairs native NVFP4 rollout with target-quantization simulation in BF16 training, then exports inference-ready low-bit weights [26]. Here alignment concerns representation and conversion semantics, while training matrix multiplications remain in BF16.

The shared information distinguishes these approaches: Jet-RL coordinates precision flow, aligned QAT emulates a representation, and TRACE communicates realized numerical decisions. Routing replay couples discrete MoE choices through a related principle [15]. TBIK illustrates the stronger requirement of coordinating reduction order across parallel configurations [28]. More specific coupling introduces dependencies on token positions, modules, and parameter versions. Agreement of the complete policies still depends on the computation surrounding the shared interface.

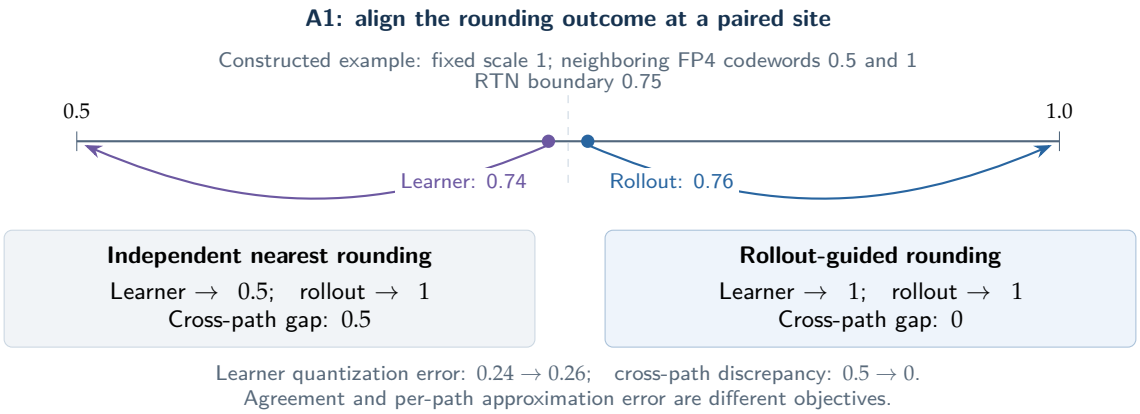


Figure 7: Constructed fixed-scale example of rollout-guided rounding using valid E2M1 codewords. The learner's approximation error increases slightly while its discrepancy from the rollout codeword vanishes. Exact guidance concerns this paired site; TRACE's compact implementation approximates the reference.

4.2. Controlling Quantization Error and Preserving Parameter Updates

Alignment alone does not ensure that the sampler can express the changes made by training. For an illustrative deterministic quantizer Q, an update can satisfy

$$Q(w + \delta w) = Q(w), \tag{7}$$

even when $\delta w \neq 0$. Visibility depends on the distance to a quantization boundary and on scale behavior, as well as update magnitude. QuRL's Update-Aware Quantization uses invariant scaling to alter the relationship between quantization resolution and parameter updates [5]. Its numerical contribution is therefore distinct from ACR, which modifies the clipping rule in Section 5.2.

Figure 8 shows a smaller update crossing a boundary while a larger one stays within its cell. Changing the scale moves these boundaries.

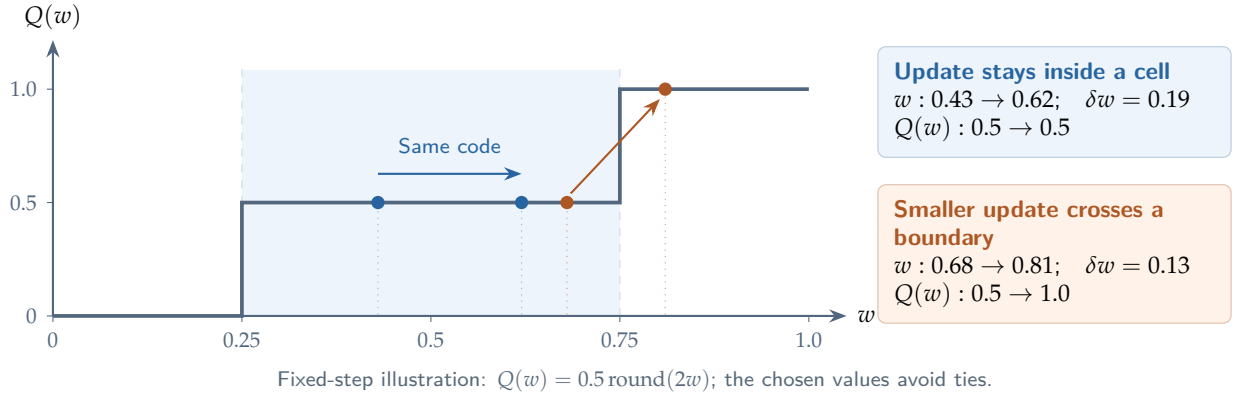


Figure 8: Quantized-update visibility in Equation (7). Under the fixed-step scalar quantizer, the blue update stays inside a cell while the smaller amber update crosses a boundary. Variable scales change these boundaries. This is a constructed illustration, not an experimental result.

QUADS addresses another residual: activation error in the rollout path. Its Residual Activation Compensation complements learner-side weight QAT, allowing alignment and error reduction to

operate on different parts of the system [8]. The component ablations are particularly useful here because they distinguish what weight alignment recovers from what activation compensation adds.

HiFloat4 / Rollout-ResQ targets rollout activation underflow through sparse residual correction [24]. Comparing learner and sampler precisions isolates a sampling-path error that raising learner precision alone does not resolve. The FP4 experiments use simulation, so they characterize learning under the simulated arithmetic; native-FP4 acceleration remains unmeasured.

FP8-RL maintains numerical fidelity as the policy changes through blockwise W8A8 rollout, scale recalibration for FP8 attention and KV execution, and dynamic weight synchronization [4]. It integrates these components with adopted TIS/MIS correction. Its v2 experiments also explore end-to-end FP8, so learner precision varies across the paper's experimental settings.

Miles exposes fidelity choices within an integrated precision recipe: selected tensors remain in BF16, and an optional dequantized backward path trades training throughput for gradient stability without changing the quantized forward values [25]. Such exceptions belong to the numerical configuration; they do not imply that every operation uses the headline format.

KV4-RL allocates fidelity across positions instead of raising precision uniformly [17]. INT4 KV storage is supplemented by bounded INT8 protection for attention-sink and recent positions. Its retained TIS recipe makes the distinction between sampler fidelity and conditional correction especially clear: protecting the cache changes the data-generating computation even when the correction is held fixed. The experiments use colocated, quota-based partial collection, with generation and optimization alternating; responses may span collection boundaries.

These methods preserve different information needed by learning. Parameter rescaling protects update visibility, residual compensation reduces activation error, recalibration follows distributional change, and positional protection targets sensitive cache states. Their W/A/KV targets locate the computation on which each mechanism acts.

4.3. Residual Mismatch and Limits of Alignment

Alignment and fidelity constrain different comparisons: agreement between endpoints and faithfulness to a higher-precision computation. Shared weights can coexist with different activations, and a common quantizer with different surrounding kernels. Each result is therefore conditional on the shared interface, parameter snapshot, and input history.

Residual differences connect numerical design to learning correction. A more faithful sampler may still lag behind the learner, and conditional agreement at observed prefixes does not restore a response distribution changed by earlier decisions. A learning intervention addresses how these remaining differences affect estimation or the resulting update.

Maintaining a shared numerical interface requires information as well as arithmetic. Operator choices require coordination, QAT depends on matching quantizer semantics, and rounding guidance requires storage and correspondence with learner states [3, 6, 8, 13]. Fidelity control instead uses calibration, compensation, or selective execution to preserve sensitive information [4, 5, 17]. Table 4 compares these dependencies and the discrepancies each leaves unresolved.

The relevant correction depends on which residual effect changes the estimator or update, and on the information available to control it.

Table 4: Numerical strategies, the information they share or preserve, and their remaining execution differences.

Strategy	Shared or protected object	Information dependency	Residual discrepancy
Precision flow	Corresponding operator precision and granularity	Coordinated numerical configuration	Engine and surrounding-kernel differences.
Aligned QAT	Learner representation corresponding to rollout	Matching quantizer semantics	Native versus simulated arithmetic; unmatched activations.
Guided rounding	Realized rollout quantization decisions	Stored and matched rollout guidance	Unshared modules and downstream propagation.
Fidelity control	Scales, update resolution, residuals, or protected states	Calibration, compensation, or selective execution	Other execution paths and policy staleness.

5. Learning under Residual Mismatch

Residual mismatch enters learning through both estimation and optimization. Estimator correction changes how behavior samples represent a target expectation; update control regulates their influence; scheduling changes the scale of adaptation over time. These objectives lead to different trade-offs among bias, variance, progress, and stability.

5.1. Correcting Gradient Estimation

Multiplicative correction and truncation. Importance sampling supplies a conditional change of measure under the assumptions of Equation (2). Practical surrogate objectives usually distinguish the mismatch correction from the proximal update ratio. With frozen p_{prox} and recorded behavior q, define

$$c_t = \frac{p_{\text{prox}}(a_t \mid h_t)}{q(a_t \mid h_t)}, \qquad r_t(\theta) = \frac{p_\theta(a_t \mid h_t)}{p_{\text{prox}}(a_t \mid h_t)}. \tag{8}$$

A representative decoupled surrogate is

$$\widehat{L}(\theta) = \frac{1}{N} \sum_t \text{sg}(\bar{c}_t) \min[r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t], \tag{9}$$

where $\bar{c}_t$ is a possibly truncated correction, the sampled advantages are held fixed, and sg denotes no differentiation through the correction weight. The separation exposes two different operations: bounding the mismatch correction c_t and clipping the proximal ratio r_t. Individual objectives differ in how they perform and combine these operations. An upper-truncated correction, for example, takes $\bar{c}_t = \min(c_t, C)$, whereas masking can discard selected ratios. Here N counts valid sampled tokens. Response-level averaging and length normalization assign different relative weights, so the reduction rule is part of the estimator being compared.

FlashRL's truncated importance sampling (TIS) provides a basic bounded correction using actual rollout probabilities [1]. MiniRL exemplifies the mismatch-aware distinction between behavior and proximal policies underlying this notation [14]. FP8-RL adopts token-level TIS and masked importance sampling (MIS) variants, while KV4-RL retains TIS alongside cache protection [4, 17]. The

system studies show how the correction operates alongside other numerical components. Truncation bounds extreme influence while changing the expectation; recording an actual behavior denominator supplies the required probability semantics, but does not remove that truncation bias.

Adapting the amount and location of correction. Adaptive importance sampling (AIS) uses observable mismatch diagnostics to interpolate between uncorrected and fully importance-weighted gradients [7]. Its adaptive quantity is the estimator mixture, not a precision schedule. Calibrated importance sampling (CIS) instead derives a confidence-dependent importance-weight cap from a discrepancy representation [20]. These approaches differ in what they adapt: overall correction strength versus the location of truncation across token confidence. CIS is used here as general TIM evidence; its BF16 cross-engine experiments are not direct FP4 validation.

Additive score correction. Score Centering addresses a different object from the sampled-token importance weight [10]. Here the *score* is the parameter-gradient vector $s_\theta(a,h) = \nabla_\theta \log p_\theta(a \mid h)$, not the reward assigned to an answer. Its mean is zero under the learner distribution, but can be nonzero under the actual behavior distribution. In Equation (5), that conditional mean produces an additional term proportional to the conditional mean advantage. Group-centered rewards need not remove it because prompt-level centering does not center the signal at every generated prefix.

Score Centering replaces the score at a prefix with

$$\widetilde{s}_\theta(a,h) = s_\theta(a,h) - \mathbb{E}_{b\sim q(\cdot|h)}[s_\theta(b,h)]. \tag{10}$$

With the exact expectation, its mean under q vanishes and the drift term in Equation (5) is removed [10]. The remaining covariance is still measured under q, so this is not full recovery of an on-policy gradient. The reported implementation uses sampler top-k log-probabilities and models the omitted tail using the learner distribution; its default is $k = 128$ [10]. The omitted mass therefore enters through a modeled tail rather than being discarded. The coefficients used to form the correction are detached, and its fidelity depends on that behavior reconstruction.

Figure 9 gives a one-parameter example. Changing the gradient's conditional mean does not require changing the sampler or forcing the learner probabilities to equal it. This separates a correction to the estimator from direct numerical alignment. When combined with importance weighting, the centered object must be the corresponding weighted score; an arbitrary weighted loss with the original unweighted center need not have the intended cancellation [10].

The covariance analysis of baselines and score centering makes estimator granularity central to their composition [27]. For the sequence-level regularized importance estimators studied there, the group-mean baseline already supplies the centering benefit. At token level, the same baseline generally leaves bias under regularized ratios. The useful centering construction therefore depends on the estimand, weighting, and reduction rule.

These corrections differ in their targets and information requirements. Ratio-based methods use sampled-action probabilities to adjust relative weights; score centering uses a behavior-weighted score expectation to remove conditional drift. Truncation and tail reconstruction approximate different objects. Their gradient interpretation also depends on the learner computation, including any surrogate gradient through a quantizer. These distinctions remain relevant when corrections are combined.

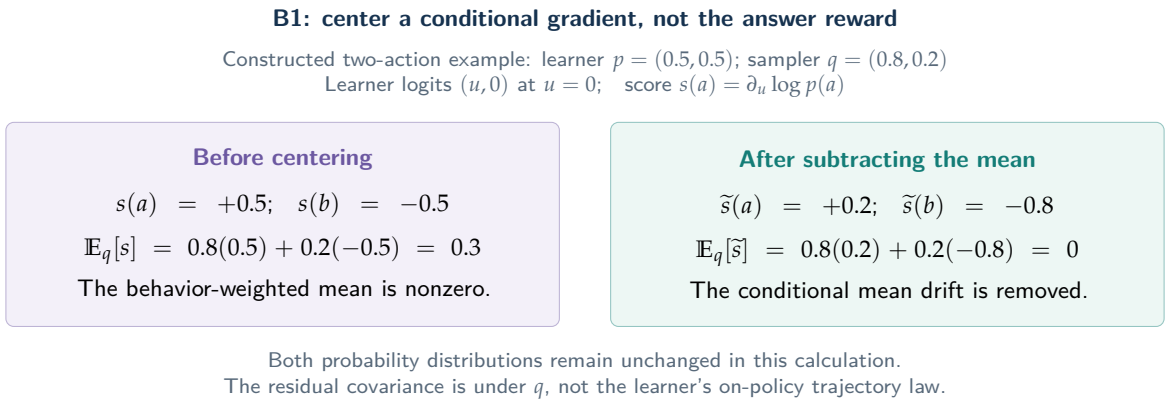


Figure 9: Exact score centering in a constructed scalar-logit example. Sampler probabilities weight the derivatives in Equation (10). Centering changes the conditional gradient mean while leaving both distributions fixed; practical top-k reconstruction adds an approximation.

5.2. Controlling the Influence of Policy Updates

Restoring or reshaping update constraints. QuRL's Adaptive Clipping Range (ACR) adjusts the update boundary in response to the relationship between proximal and behavior probabilities [5]. QaRL's Trust-Band Policy Optimization (TBPO) combines negative-sample treatment with sequence-level trust-band control [6]. The full-pipeline FP8 method first matches the lower clipping quantile on negative-advantage tokens to a BF16 reference, then adjusts the upper boundary to rebalance positive and negative contributions [11]. Periodic reference passes amortize the calibration cost. All three act on the update constraint, but use different signals and aggregation levels. "Calibration" in the last method refers to objective boundaries, not quantizer scales.

The shared issue is which learning signal reaches the parameter update. A token-level boundary can restore a contribution lost to clipping, whereas a sequence-level trust band controls a whole response. Widening either region admits more updates, but their value depends on advantage sign and influence. The clipping count becomes informative when connected to those retained contributions.

Batch reliability and divergence geometry. P3O uses normalized effective sample size (ESS) of the batch's policy ratios to adapt a score-weight cap and off-policy regularization [21]. Batch reliability thus determines how strongly the update is constrained. Its FP8-rollout experiment studies numerical mismatch directly, while the broader comparisons show dependence on the model and collection setting. The adaptive object differs from AIS: the estimator mixture in AIS, and update constraints in P3O.

DPPO replaces sampled-ratio clipping with a divergence-based trust region [29]. DRPO retains the Binary-TV geometry but replaces hard masking with a smooth regularizer that supplies corrective gradients outside the boundary [22]. DPPO provides the broader trust-region foundation; DRPO evaluates both FP8-rollout-only and end-to-end FP8 MoE settings. The shared object is the geometry of the permitted update, while the available low-precision evidence differs.

Controlling disagreement through update direction. ACRL uses advantage-guided probability adjustment, with response-level discrepancy relative to a reference controlling token reweighting

[9]. Its target is how updates change disagreement, rather than the recovery of a distributional expectation.

TRIAGE connects the advantage sign to the learner–sampler log gap [12]. Using $d_\theta = \log p_\theta - \log q_k$ in this survey's notation, the product $A_i d_{\theta,i,t}$ identifies the sign of a simplified token's own mismatch-energy contribution. Cross-token coupling and optimizer history can change the realized motion, so this is a local directional indicator. Both $A_i > 0, d_\theta > 0$ and $A_i < 0, d_\theta < 0$ are locally amplifying. TRIAGE intervenes selectively on the latter: increasing the probability of a reward-positive response can be legitimate learning even if its gap grows.

The gate first diagnoses 64-position segments of negative-advantage responses from their mean gap and deep negative tail. Within an eligible risky segment, only negative-gap tokens receive an attenuated raw weight $b_{i,t}$. The weighted response loss is normalized by the sum of its weights. Relative to ordinary response averaging, its effective multiplier is therefore $b_{i,t}/\bar{b}_i$, not simply $b_{i,t}$. Normalization reallocates contributions: for illustrative raw weights $(0.3, 1, 0.3, 1)$, the effective multipliers are approximately $(0.46, 1.54, 0.46, 1.54)$. If every weight is equal, the normalization cancels it. This controls relative influence, not the total gradient norm.

Repair acts on positive-advantage responses with sufficiently negative mean gaps in separate 128-position segments. A one-sided pseudo-Huber penalty adds upward pressure on learner log probabilities. Its derivative bounds the additive corrective coefficient; it does not guarantee a bounded full gradient or contraction for every token. All valid tokens in an active repair segment share that coefficient, including any positive-gap tokens. Gating and repair thus modify different signed contributions rather than uniformly penalizing disagreement [12]. The evaluated recipe preserves native W4A4 forward execution on both endpoints. The documented KV cache is BF16, and the paper reports no KV4 experiment. Applying the same control to KV4 is a transfer hypothesis, not an established result.

Accepting whole updates in the inference policy. MIPU constructs candidates relative to the sampler, then assesses whether a synchronized candidate should become the next inference policy [23]. This separates candidate construction from acceptance after numerical conversion. The practical inference-gap proxy supports acceptance or rollback in the evaluated FP8 runs; a general guarantee of monotonic improvement would require stronger conditions.

These controls span tokens, segments, responses, batches, and whole candidate updates. Local rules can reveal concentrated risk but rely on noisier statistics; aggregation improves statistical reliability while obscuring its location. Whole-update acceptance adds a further distinction: a favorable learner-side candidate can still be rejected after conversion to the inference policy.

5.3. Adapting the Optimization Schedule

A complementary intervention changes the scale of learning over time. Beyond Precision motivates learning-rate scheduling through the evolution of optimization and mismatch [18]. This is complementary to changing a numerical path or a token weight: it modifies the trajectory of parameter updates themselves.

Schedule adaptation remains less developed for direct low-bit settings than the estimator and update controls above. Its promise depends on preserving progress as well as stability: smaller

steps can avoid divergence while delaying useful learning. Comparable quality within a comparable budget is therefore the relevant test of transfer to aggressive quantization.

5.4. Compatibility, Bias, and Failure Modes

The operators in Figure 10 can share a reweighting implementation while targeting different objects. Importance correction targets an expectation; a gate, trust band, or acceptance rule regulates the update that reaches the next policy. Composition requires a common behavior law and a definition of the estimator after all operations have acted.

Aggregation is part of that definition. The baseline analysis shows why token-level and sequence-level regularized estimators can require different centering constructions [27]. The loss, weighting, and reduction rule together determine whether a component addresses the intended residual bias.

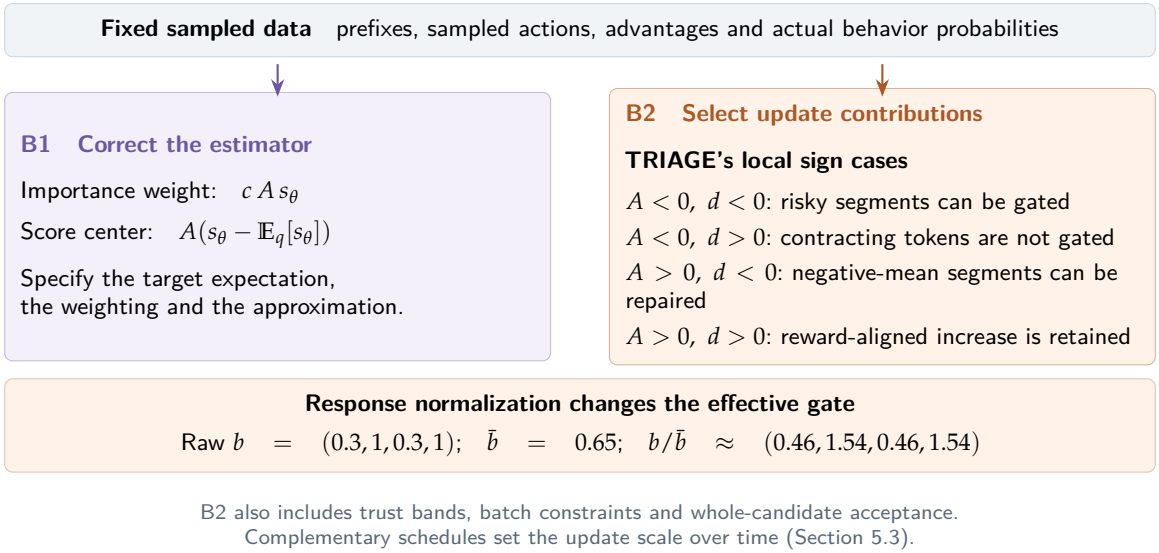


Figure 10: Estimator correction and update control. TRIAGE's sign cases use $d = d_\theta$ together with segment conditions. Illustrative raw weights show how response normalization changes effective influence; they do not represent a diagnosed segment. Shared-parameter interactions limit token-wise directional predictions.

Composition appears at several levels. QaRL and QuRL combine numerical and learning interventions; QUADS couples alignment with activation-error compensation. Score Centering combines estimator corrections by centering the importance-weighted score [5, 6, 8, 10]. Each combination addresses residual effects left by its other components.

That complementarity must be established for the joint estimator. A center computed for an unweighted score need not remove drift after weighting; successive masks can discard the same examples; reference passes add computation. Consistent behavior probabilities and parameter snapshots supply a common basis, while the retained signed contributions and total cost determine whether the combination improves learning.

6. Evidence Synthesis and System Trade-offs

Within-study comparisons connect numerical execution to update behavior and learned-policy quality. They expose how closely the endpoints share computation, how samples enter the estimator, and

which contributions survive optimization. These connections explain why numerical agreement, stability, and quality can respond differently to an intervention. Training cost determines whether the resulting benefit improves learning efficiency.

6.1. Experimental Regimes

Asymmetric learner–sampler precision exposes differences between numerical representations. Low precision on both sides makes the consistency of their execution paths and the resulting update especially salient. KV quantization acts on retained state, whereas weight and activation quantization alter other parts of computation. Low-rank adaptation and full-parameter RL also provide different degrees of freedom for responding to these changes.

AIS evaluates diffusion-based LLaDA alongside autoregressive Qwen models [7]. Its autoregressive experiments connect directly to the token-level formulation used here. Interpreting the denoising results requires likelihoods defined for that generation process, so those experiments broaden the empirical scope without validating the autoregressive factorization for diffusion.

Evaluation precision answers a separate question from training precision. A common BF16 evaluator, as used in KV4-RL, compares the parameters learned from different rollout distributions while holding the evaluation path fixed [17]. Evaluation through the intended low-precision sampler also includes the numerical realization of those parameters, as in TRACE's FP4 policy results [13]. Both are informative, but they should not be pooled as the same measure of policy quality.

Architecture and collection protocol further condition conclusions. MoE execution introduces discrete route choices; cache-heavy long responses expose different runtime bottlenecks from shorter generations; partial collection permits a response to contain tokens sampled under different parameter versions. A method evaluated in one of these regimes may motivate a hypothesis in another, but that transfer is not itself an experimental result.

Miles adds production-scale agentic system evidence, whereas QATFactory isolates a smaller NVFP4 QARL workload [25, 26]. Native low-precision arithmetic, simulated quantization, and BF16 execution on quantized operands are distinct regimes. In particular, HiFloat4 reports simulated FP4 RL rather than measured native-FP4 training [24].

Table 6 summarizes these dimensions and the limits of their coverage.

Table 5 makes four contrasting configurations explicit. A row refers to a reviewed experimental setting, not every possible implementation of the method. In particular, low-bit rollout does not establish native low-bit learner execution, and a cache-only intervention does not establish joint weight/activation quantization.

Table 8 brings together four comparison cases that illuminate these interactions. Appendix A covers all twenty-nine focal works, and the companion evidence collection provides source locations for observations from all nineteen core studies.

6.2. Findings on Stability and Learning Quality

Across these regimes, the evidence links the interface being aligned to the estimator, retained update contributions, and subsequent learning. Figure 11 maps two directly reported comparison sets. A shared method name in separate studies does not make their experiments transitive: model, execution, objective and checkpoint rules still determine what each comparison establishes.

Table 5: Representative execution configurations, with numerical scope retained. QAT describes an adaptation procedure; its arithmetic must be stated separately.

Study	Learner path	Rollout and collection setting
TRACE [13]	Guided fake-quantized forward; BF16 backward.	NVFP4 W4A4 on routed experts and KV4 in softmax attention; other modules BF16. Asynchronous collection.
TRIAGE [12]	Native NVFP4 W4A4 forward; not a claim that all training states are FP4.	Native W4A4 forward; BF16 KV in the documented characterization. Synchronous math RL; no reported KV4 experiment.
Score Centering [10]	BF16 learner; distributional reconstruction supplies the correction.	Quantized-sampler stress tests include FP4 KV and INT8 sampler with INT4 KV. Policy-lag tests form a separate setting.
KV4-RL [17]	BF16 learner.	INT4 KV with bounded INT8 protection. Quota-based partial collection; generation and learning alternate. This is a cache-focused experiment.

Table 6: Coverage matrix for the qualitative snapshot. Rows are independent dimensions; the presence of two regimes does not imply that their combination has been evaluated.

Dimension	Represented in the focal collection	Boundary of the synthesis
Intervention	A1/A2 numerical design; B1/B2 learning rules; complementary scheduling.	Direct low-bit evidence is thinner for B3 than for the other classes.
Numerical interface	Weight/activation quantization, KV storage, scales, rounding, routing, and synchronization.	An interface-level result does not establish full-network agreement.
Precision	FP8, FP4/NVFP4, INT8/INT4; BF16/FP16 comparisons provide context.	Format labels omit scale granularity, precision exceptions, and native versus simulated arithmetic.
Model and adaptation	Dense and MoE models; low-rank and full-parameter updates appear in different studies.	Architecture and adaptation are separate axes; cross-regime transfer is not assumed.
Collection	Snapshot-based hybrid execution, policy lag, and quota-based partial collection.	Stateful provenance and fully overlapped execution require additional behavior and timing semantics.
Learning and generation	PPO/GRPO-style autoregressive RL; AIS also studies diffusion generation.	This survey's likelihood equations are autoregressive; denoising requires its own formulation.
Contribution role	Nineteen core studies, nine supporting studies, and one exploration boundary.	Role identifies relevance to the argument, independently of experimental strength.

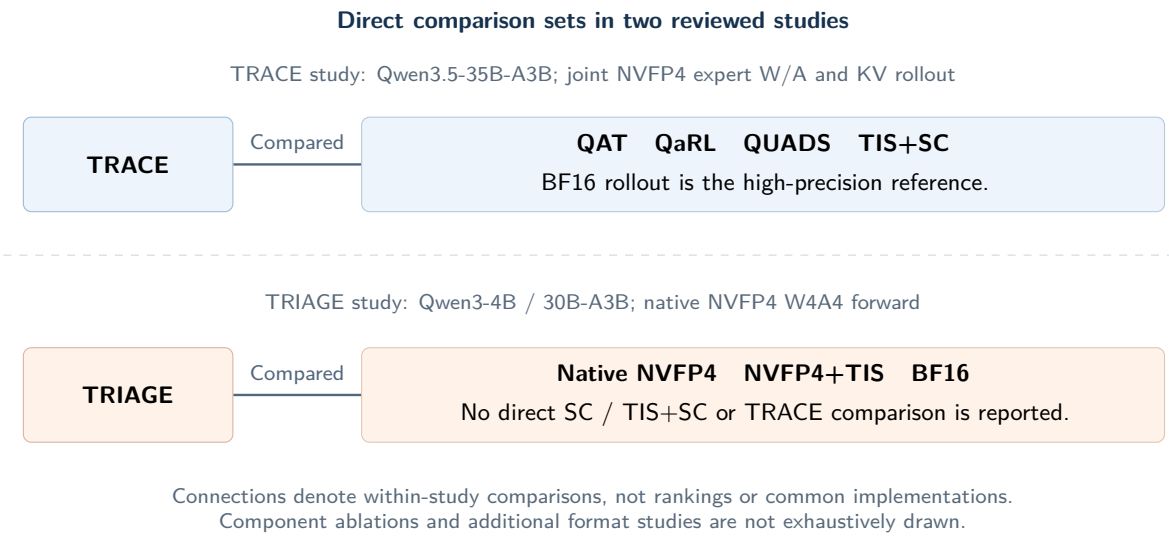


Figure 11: Selected comparison sets in TRACE and TRIAGE [12, 13]. The figure exposes available and missing direct comparisons while retaining their distinct settings. It establishes no cross-study performance order. Checkpoint rules for the TRACE–SC comparison are given in Table 8.

Reading learning trajectories. The common training diagnostics measure different objects. Policy entropy describes the spread of the next-token distribution; training reward describes the scores assigned to sampled answers; a mean absolute log-probability gap describes agreement on sampled tokens. Table 7 connects these quantities to the conclusions they support. Their reduction rules and measurement policies matter: a response mean and a token mean weight the same batch differently, while a current learner–behavior gap can include policy lag as well as numerical mismatch.

Table 7: Interpreting training diagnostics. Formulas illustrate common definitions; they do not reconstruct undocumented logging code. Valid-token and response reductions must be stated.

Quantity	What it measures	What it does not establish
Policy entropy H	$-\sum_a p(a) \log p(a)$ for a specified next-token policy, subsequently aggregated over prefixes.	A lower value need not indicate better learning. A label such as *entropy loss* does not establish that entropy regularization is active.
Mean reward $\bar{R}$	$B^{-1} \sum_i R_i$ over B sampled responses. With strictly binary correctness reward, it is the sampled correctness rate.	It is not the group-normalized advantage, a held-out benchmark score, or an invariant scale across reward functions.
Mean absolute gap G	$N^{-1} \sum_t \lvert d_{\theta,t} \rvert$ over N valid sampled tokens: the magnitude of learner–behavior log-probability disagreement.	It removes sign and local concentration; it is neither KL divergence nor proof of estimator fidelity or policy quality.

TRIAGE's 30B experiment illustrates the distinction: TIS keeps the aggregate gap from diverging over the measured horizon, yet its later reward and downstream scores remain below the full method [12]. Conversely, useful reward-driven updates can increase some signed gaps. Stability must therefore be read jointly with learning progress and the retained signed contributions, rather than inferred from a single discrepancy curve.

Correction and mismatch evolve together. QuRL observes disagreement beyond an initially stable period, with ACR and UAQ addressing different changes in clipping and update visibility [5]. AIS adapts the correction mixture, ACRL regulates discrepancy through update influence, and Score Centering removes a conditional estimator drift term [7, 9, 10]. Each intervention changes the policy and hence the mismatch encountered later. P3O's Qwen3-8B FP8 comparison similarly links batch-adaptive control to preserved later performance within the evaluated horizon [21]. Such trajectory evidence supports stability in the reported setting, rather than a universal safeguard against collapse.

Alignment gains depend on the shared interface. Jet-RL couples precision flow; QaRL, QUADS, and TRACE couple representations or realized rounding decisions [3, 6, 8, 13]. QUADS separates two effects: asymmetric weight QAT recovers much of the performance loss, and rollout activation compensation further narrows the gap. Its asymmetric-versus-symmetric comparison shows why nominal symmetry alone does not define effective alignment. HiFloat4's simulated precision ablations also locate a residual error in rollout activations [24]. FP8-RL follows a changing policy through calibration and synchronization, while QATFactory compares learning in the target representation with quantizing only the learned endpoint [4, 26]. These comparisons make the shared interface and remaining error more informative than the nominal format alone.

Table 8: Within-study evidence and its comparison boundary. Values belong to the stated setting; they are not pooled across studies.

Study	Controlled comparison	Reported result	Interpretation and limit
TRACE [13] 35B MoE	Joint NVFP4 W/A/KV rollout; TIS+SC uses cap 2 and top-128 reconstruction.	Four-task mean: TRACE 75.3, TIS+SC 74.1, BF16 74.9.	SC selects its best mean checkpoint through 400 steps. TRACE does not win on every task; this table reports no uncertainty interval.
TRIAGE [12] 30B MoE	BF16, native NVFP4, TIS, and full gating/repair recipe. Stable arms use step 1700.	Five-task mean: TRIAGE 70.96, TIS 63.18, BF16 72.41.	Supports recovery in this native-W4A4 setup; no SC/TIS+SC or KV4 comparison is reported.
Score Centering [10] 30B MoE	Correction rules share the objective; quantization and lag are tested separately.	With FP4 KV, SC and TIS remain stable; stronger INT8/INT4 stress separates them further.	Quantization curves report training accuracy. Shared-objective controls are not full reproductions of every baseline recipe.
KV4-RL [17] 8B dense	Cache precision varies with TIS retained; all learned policies are evaluated in BF16.	At 400 updates, six-task mean: 52.8 versus BF16's 52.4; cumulative full-iteration speedup 1.27×.	Evaluation-seed variation is not independent training-run variation; this is not a joint W4A4/KV4 comparison.

Signed update selection explains effects hidden by averages. The full-pipeline FP8 study connects excessive lower-side clipping of negative-advantage tokens to pathological training dynamics, whereas TRIAGE uses local and direction-dependent information to select interventions [11, 12]. DRPO preserves corrective gradients outside its trust-region boundary; MIPU instead tests a whole candidate after conversion to the inference policy [22, 23]. Together these methods show why the location, direction, and stage of control matter. Aggregate clipping fractions or mean log-probability gaps can conceal these distinctions. TRIAGE's MoE routing-conditioned analysis further separates a population trend from reliable directional prediction for each token. Checkpoint selection adds another boundary: the full-pipeline FP8 study reports its main GRPO endpoints from the best average checkpoint over 500 steps [11], whereas TRIAGE evaluates stable arms at their final scheduled step. Similar endpoint scores can therefore summarize different trajectory-selection rules.

The TRACE–SC comparison is particularly informative because it holds the joint low-precision rollout setting fixed while changing the correction approach. Its 1.2-point mean difference is accompanied by mixed task-wise outcomes and a best-checkpoint rule for SC. TRIAGE establishes a different result: it recovers much of the loss left by TIS in native W4A4 execution, but does not resolve whether gating and repair outperform score centering in that regime. These are distinct evidence boundaries, not grounds for an overall ranking [12, 13].

These studies illustrate different forms of complementarity. QUADS combines two numerical interventions, QuRL combines update visibility with clipping control, and KV4-RL improves cache fidelity while retaining the correction recipe. Their results connect the component being changed to the contributions available for learning. They support the combinations tested within each system and suggest how numerical fidelity and learning correction can address different residual effects.

6.3. Efficiency Gains and Implementation Costs

An intervention's computational benefit depends on the work required for generation, learning, synchronization, and correction. For serialized phases, iteration time can be decomposed as

$$T_{\text{iter}} = T_{\text{rollout}} + T_{\text{learner}} + T_{\text{sync}} + T_{\text{quant/calib}} + T_{\text{other}}, \tag{11}$$

where synchronization and quantization/calibration costs are separated from generation and learning. Accelerating one phase exposes the others. With overlapping execution, the relevant quantity is the critical path, so the phase times in Equation (11) cannot simply be added.

The serialized model also gives a simple bound on what a rollout improvement can achieve. If rollout occupies fraction f of the baseline time, a rollout speedup s and added work hT_0 give $T/T_0 = (1-f) + f/s + h$, holding the other phases and workload fixed. Figure 12 illustrates the arithmetic. Overlap, changed response lengths, or changed concurrency require a different cost model.

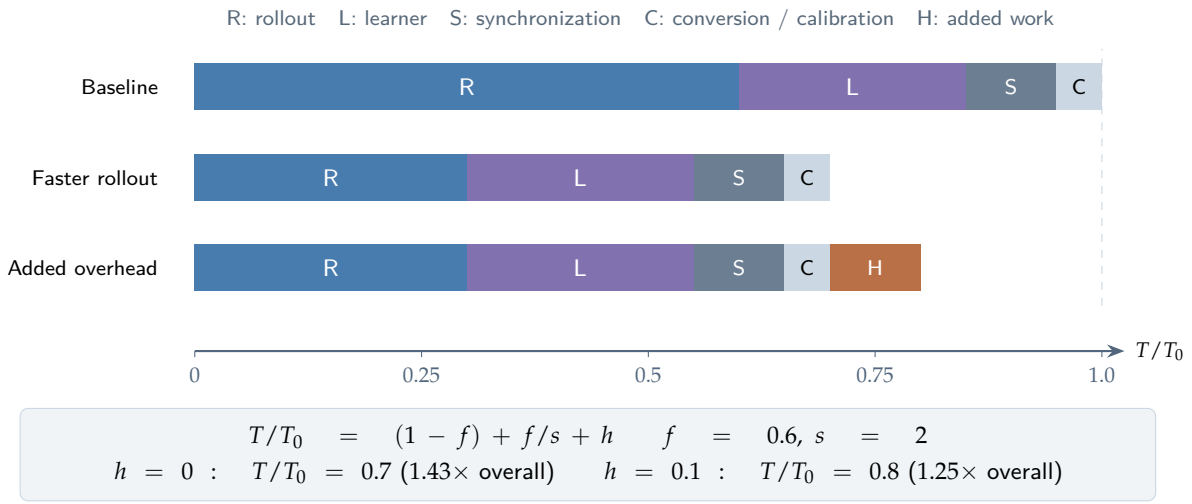


Figure 12: Serialized timing with a faster rollout phase and added overhead. Doubling rollout speed reduces iteration time from T_0 to $0.7T_0$ when rollout accounts for 60% of the baseline. Added work of $0.1T_0$ reduces the overall speedup from 1.43× to 1.25×. This is a constructed illustration, not an experimental result.

Each intervention changes a different part of this cost. FP8-RL illustrates the work needed for scale estimation and numerical conversion [4]. Reference calibration adds learner computation; guided rounding adds storage, transport, and state matching; selective precision adds work in the protected region. Table 9 relates these dependencies to the information each method needs.

P3O requires batch-level ratio statistics, while MIPU's acceptance step adds inference-side assessment and possible rollback [21, 23]. Miles and QATFactory emphasize conversion and inference-ready weight transfer as part of the execution contract [25, 26]. TBIK's coordinated reductions can carry a kernel-performance cost [28]. Rollout-ResQ's sparse correction is designed for efficient hardware execution, but its simulated experiments do not measure a native FP4 speedup [24].

Token throughput also depends on the realized workload. A policy that produces shorter responses can appear faster without improving execution per token; one that produces longer responses can incur more work despite a faster kernel. Memory savings can increase concurrency, which changes the workload seen by attention and compute kernels. Serving research such as

PagedAttention provides background for why cache management and scheduling affect observed throughput [32], but serving speed alone does not establish RL learning efficiency.

Table 9: Information and execution dependencies of the reviewed approaches. These interfaces explain system cost without assigning an overhead ranking.

Approach	Information or execution dependency	Consequence for implementation
TIS / AIS / CIS	Behavior probabilities and learner/proximal probabilities	Stored probabilities must include the sampling transformations.
Score Centering	A behavior-weighted score expectation, exact or approximated	Exact evaluation requires distributional information; reconstruction introduces approximation.
Aligned QAT / shared precision	Matching quantization and conversion rules across endpoints	Precision exceptions, kernels, and weight export form part of the configuration.
Guided rounding	Realized rollout-side quantization information	Guidance requires storage, transport, and matching to learner states.
Calibrated Clipping [11]	Higher-precision reference ratios at calibration points	Reference passes add training computation, amortized across calibration intervals.
Batch-adaptive control [21]	Policy-ratio statistics aggregated over the batch	Reduction rules and the sampled batch determine the reliability estimate.
Inference-policy acceptance [23]	Assessment of the synchronized candidate	Candidate evaluation and possible rollback add work beyond learner updates.
Selective KV precision	Low-bit cache execution with protected positions	Protected positions add work whose share depends on the context workload.

KV4-RL illustrates the difference between generation and training gains. Over 400 iterations on Qwen3-8B-Base, it reports 1.80× cumulative rollout speedup and 1.27× cumulative full-iteration speedup over BF16 rollout [17]. Other phases absorb part of the generation gain. Its common-BF16 evaluation separately compares learned-policy quality under a shared evaluation precision. Both conclusions apply to the study's collection workload.

Time or compute to a specified quality level connects these measurements to the learning objective. Endpoint scores, iteration time, and rollout throughput supply different parts of the picture. The trajectory is needed to recover time-to-quality when an intervention changes response length, learning speed, or checkpoint selection.

6.4. Evidence Limits and Coverage Gaps

The strongest empirical support concerns mechanisms isolated within a specified setting. Model families, tasks, horizons, hardware, and kernels vary across studies. Mathematical reasoning is a recurring evaluation task; interactive trajectories and broader task distributions are less directly covered. Transfer across formats depends on scaling rules, precision exceptions, and surrounding kernels.

Source-linked observations expose the basis of each comparison, while unreported implementation conditions remain unresolved. This curated narrative synthesis does not establish exhaustive coverage or independently reproduce the reported experiments. Seed counts, uncertainty, negative results, and cross-hardware replication are not systematically extracted across the full corpus. Where a selected comparison reports uncertainty, its source of randomness must remain explicit:

repeated decoding at fixed learned parameters does not quantify variation across independent RL training runs. Similarly, a smoothed trajectory is not a confidence interval unless the source defines it that way.

Direct low-bit scheduling evidence remains limited, and stateful provenance is only partly characterized. Multi-turn system demonstrations provide relevant evidence, but controlled attribution across long interactions remains limited. Diffusion results extend the empirical context while requiring a likelihood formulation beyond the autoregressive analysis used here.

7. Open Problems

The remaining challenge is to predict when mismatch becomes harmful, how interventions interact, and which benefits transfer across systems and tasks.

7.1. Predicting Harmful Mismatch

Learning depends on where error occurs and which update it changes. Confidence-conditioned tails, signed clipping distortions, conditional drift, and directional concentration are candidate diagnostics with closer links to the update than mean disagreement. Their ability to predict instability or lost progress early enough to guide intervention remains unresolved.

Collapse, reduced progress per update, and failure to reach a quality threshold need not share a precursor. A mechanism-based diagnostic should remain predictive across models and correction rules. Lowering it without improving learning may instead reflect correlation, or suppression of useful signal alongside the measured risk.

7.2. Composing Numerical and Learning Interventions

Greater fidelity alters the discrepancy encountered by a correction; restrictive update rules can conceal the value of finer quantization by suppressing the affected contributions. A component's value therefore depends on its marginal effect on learning progress and cost after other interventions act.

Multiple masks, truncations, or trust bands can suppress the same contributions, making weak learning appear stable. Analysis of the joint estimator must identify its bias and variance, the contributions retained from positive and negative advantages, and the cost of reference passes or reconstructed expectations. This connects compatibility to quality gained per unit of computation.

7.3. Precision, Staleness, and Long-Horizon Interaction

Partial and asynchronous collection can make the sampling version token-dependent. A probability gap may then combine parameter change with execution mismatch, as Equation (1) makes explicit. Separating their effects requires information about snapshots, replay, or retained cache states that a production system may not store. The smallest behavior record sufficient for reliable correction remains a joint statistical and system question.

Long responses amplify this difficulty. An early numerical perturbation changes the prefixes visited later, even if conditional correction at each observed prefix is accurate. This suggests allocating precision according to position, layer, context, and training phase, with learning benefit as the objective. Local tensor error alone cannot identify where the extra fidelity is worth its cost.

Multi-turn interaction adds tool observations and environment transitions that shape later behavior. These may need to accompany cache provenance and sampling probabilities in the behavior record. Evidence from single-response generation leaves the effectiveness of the surveyed interventions in these settings open.

7.4. Generalization and Comparable Evidence

Transfer depends on behavior and execution semantics. Correction requires probabilities of actual sampling events; rounding guidance requires correspondence between numerical states. Predecoding probabilities and changed synchronization protocols can violate these requirements. Controlled comparisons across stacks are needed to establish which benefits survive changes in kernels, collection protocols, and correction recipes.

Comparable evaluation should distinguish component controls from complete-system comparisons. The former hold execution and the base objective fixed while varying compatible corrections; the latter retain a method's essential numerical design and compare quality and total cost under matched resources. A common fixed-budget endpoint and cost to a predeclared quality target can connect statistical efficiency to runtime. Failure to reach that target remains an outcome, and a transient best checkpoint should not silently substitute for sustained progress. These principles motivate future benchmark design; this survey does not implement a new benchmark or validate a universal ranking metric.

8. Conclusion

Quantized rollout places numerical approximation inside the learning loop: the data-generating policy affects both the current update and future trajectories. The surveyed interventions address distinct parts of this loop through computation alignment, numerical fidelity, estimator correction, and update control, complemented by schedule adaptation.

Numerical fidelity changes the error presented to the learner; correction changes its influence on the update. This interaction makes average agreement, short-term stability, and rollout throughput incomplete measures of effective training. The goal is policy quality at lower total computational cost. Predictive diagnostics and joint-estimator analysis can clarify when a combination achieves that goal across systems and collection regimes.

A. Surveyed Papers and Section Mapping

Table 10 summarizes the twenty-nine focal studies, numerical regimes, and component assignments. Years denote first public appearance; bibliography records may cite later formal publications. Main discussion locations precede additional locations in parentheses. Contribution roles are defined in Appendix B.

Table 10: Surveyed papers: numerical regimes, method components, and section mapping. Conference and preprint versions count as one contribution.

Work / first year	Numerical regime	Component(s) and contribution	Discussion
Core method and system studies (19)			
FlashRL / TIS [1] 2025	Higher-precision learner; quantized rollout	**B1**: Recorded behavior probabilities and truncated correction.	**5.1** (1.1; 2.1)
Jet-RL [3] 2026	Unified FP8 learner and rollout paths	**A1; A2**: Unified FP8 precision flow.	**4.1** (6.2)
FP8-RL [4] 2026	W8A8 rollout; FP8 attention / KV; FP8 learner explored in v2	**A2; system**: W8A8 rollout, FP8 attention/KV, and stack integration.	**4.2** (5.1; 6.2; 6.3)
QuRL [5] 2026	INT8 / FP8 rollout	**B2; A2**: Adaptive clipping and update-aware quantization.	**5.2** (3.1; 4.2; 5.4; 6.2)
QaRL [6] 2026	Quantized MoE rollout; aligned learner QAT	**A1; B2**: Rollout-aligned QAT and trust-band updates.	**4.1** (5.2; 5.4; 6.2)
P3O / ESS [21] 2026	BF16 learner / FP8 rollout; dense reasoning models	**B2**: Batch-adaptive ESS cap and regularization.	**5.2** (6.2; 6.3)
AIS [7] 2026	BF16 learner / FP8 rollout; autoregressive and diffusion models	**B1**: Adaptive importance-weighted estimator mixture.	**5.1** (6.1; 6.2)
DRPO [22] 2026	BF16, FP8 rollout-only, and FP8 end-to-end MoE settings	**B2**: Smooth corrective divergence regularization.	**5.2** (6.2)
MIPU [23] 2026	BF16 learner / FP8 rollout; Qwen3-1.7B and 4B	**B2; formulation**: Inference-policy candidate construction and acceptance.	**5.2** (2.1; 6.2; 6.3)
QUADS [8] 2026	NVFP4 MoE rollout; learner weight QAT	**A1; A2**: NVFP4 MoE weight alignment and activation compensation.	**4.1** (4.2; 5.4; 6.2)
ACRL [9] 2026	FP8 rollout; discrepancy-dependent updates	**B2**: Discrepancy-dependent update control.	**5.2** (3.3; 6.2)
HiFloat4 / Rollout-ResQ [24] 2026	Simulated end-to-end HiF4 / MXFP4; rollout residual correction	**A2**: Sparse residual correction for FP4 rollout activations.	**4.2** (3.1; 6.2; 6.3)
Miles v0.1 [25] 2026	FP8 / MXFP8 / NVFP4 RL recipes; precision exceptions	**A1; A2; system**: Shared quantization, synchronization, and precision contracts.	**4.1** (4.2; 6.1; 6.3)
Score Centering [10] 2026	BF16 learner; FP4/INT4 KV stress and separate lag tests	**B1**: Additive behavior-weighted score correction.	**5.1** (2.2; 3.2; 5.4; 6.2)
Full Pipeline FP8 RL [11] 2026	FP8 learner and rollout; calibration against BF16	**B2; system**: FP8 learner/rollout and calibrated clipping.	**5.2** (3.2; 6.2; 6.3)

Continued on next page

Table 10 continued: Surveyed papers

Work / first year	Numerical regime	Component(s) and contribution	Discussion
QATFactory / QARL [26] 2026	Native NVFP4 rollout; simulated-quantization BF16 training	**A1; system**: Deployment-aligned QARL and inference-ready weight synchronization.	**4.1** (6.2; 6.3)
TRIAGE [12] 2026	Native W4A4 forward; documented BF16 KV; synchronous RL	**B2**: Direction-aware control for native NVFP4.	**5.2** (3.3; 6.2)
TRACE [13] 2026	NVFP4 expert W4A4 + softmax KV4; guided QAT, BF16 backward	**A1**: Rollout-guided FP4 rounding for MoE.	**4.1** (3.1; 6.2)
KV4-RL [17] 2026	BF16 learner; INT4 KV with bounded INT8 protection	**A2; system**: INT4 KV with selective INT8 protection.	**4.2** (5.1; 6.2; 6.3)
Supporting formulation, theory, and general TIM studies (9)			
R3 [15] 2025	MoE routing; general execution mismatch	**Support; A1**: MoE routing alignment.	**3.1** (4.1)
FP16-Mismatch [16] 2025	BF16 / FP16 numerical comparison	**Support; A2**: BF16/FP16 precision-selection evidence.	**3.1**
TBIK / Deterministic Inference [28] 2025	BF16 / FP32; matched reduction order across TP sizes	**Support; A1**: Cross-TP deterministic kernels as an alignment reference.	**3.1** (4.1; 6.3)
MiniRL [14] 2025	Hybrid-engine RL; includes FP8-rollout experiments	**Support; B1/B2**: Policy roles and mismatch-aware formulation; FP8 experiments.	**2.1** (1.2; 5.1)
Beyond Precision [18] 2026	General TIM; optimization scheduling	**Support; B3**: Optimization schedule under general TIM.	**5.3** (3.3)
DPPO [29] 2026	General LLM trust-region and cross-engine mismatch evidence	**Support; B2**: Divergence-based trust regions under general TIM.	**5.2** (3.2)
VeXact / Diagnosing TIM [19] 2026	Matched-execution diagnostic reference	**Diagnostic**: Zero-mismatch diagnostic reference.	**3.4**
CIS [20] 2026	BF16 cross-engine mismatch	**Support; B1**: Confidence-dependent correction under BF16 cross-engine mismatch.	**5.1** (3.2)
Baselines and Score Centering [27] 2026	Sequence- and token-level regularized importance sampling	**Support; B1**: Granularity-dependent roles of baselines and score centering.	**5.1** (5.4)
Exploration boundary case (1)			
QeRL [2] 2025	NVFP4 with LoRA and adaptive noise	**Boundary**: NVFP4 with LoRA and adaptive noise; exploration setting.	**Box 1**

B. Source Selection and Version Handling

Construction of this snapshot. The collection comprises twenty-nine contributions to quantized rollout, low-precision RL, and mechanisms needed to interpret training–rollout mismatch. Primary papers and official records provide the evidence. A targeted update searched arXiv metadata through October 20, 2026, followed by primary-text checks and backward citation tracing. This curated narrative review does not claim exhaustive coverage.

Inclusion and exclusion. A focal study contributes a direct quantized-rollout or low-precision RL intervention, a mechanism needed to interpret that setting, or a carefully identified boundary case. General quantization and RL background is cited selectively. Papers devoted only to low-precision pretraining, unrelated generation modalities, or alternative rollout acceleration do not become focal entries solely because they use the same precision format.

Versions and names. The catalog distinguishes first-publication dates, publication venues, and reviewed versions. QeRL, QuRL, and QaRL use formal publication records in the bibliography. Conference and preprint versions, aliases, and changes in presentation count as one contribution. Unpinned arXiv links are distinguished from locally reviewed versions, since a later revision may change the claims supported by a source.

Attribution of evidence. Method claims and observations are attributed to primary sources. The likelihood, drift, and clipping identities explain mechanisms under the stated assumptions; they are distinct from experimental findings. Figures and tables are original syntheses, and constructed examples are labeled separately from published measurements. The companion evidence collection links observations to reviewed sources.

Evidence roles and verification. Contribution role identifies relevance to the argument; evidence domain identifies the setting examined. Neither grades experimental quality. Source-linked observations retain their conditions and limitations. No independent reproduction was performed, and unextracted statistical or implementation fields remain unassessed. The catalog separates identified reviewed versions from version-unspecified records.

C. Evidence, Assumptions, and Transfer

Transfer depends on the relationship between sampled behavior and learner computation. Parameter versions, numerical paths, decoding, and retained state determine the behavior law; learner, proximal, and reference policies define separate loss roles. Changing execution can alter a correction's meaning even at the same nominal precision.

Each approximation leaves a specific residual: prefix selection after conditional correction, truncation bias, unmatched execution after alignment, or reconstruction error in an expectation. A tested combination supports its reported estimator and regime; transfer requires the assumptions responsible for that result to remain valid.

D. Terminology

Table 11: Terminology and component abbreviations used in the text and figures.

Term	Meaning
TIM	Training–inference mismatch; the survey emphasizes the learner–rollout interface.
W / A / KV	Weights, activations, and key–value cache; distinct numerical targets.
MoE / LoRA	Mixture-of-experts architecture / low-rank adaptation; architecture and adaptation are separate dimensions.
FP8 / FP4 / NVFP4	Floating-point format families with eight or four bits; NVFP4 is a particular four-bit representation. Scaling, accumulation, and exceptions also determine execution.
INT8 / INT4	Integer quantization at eight or four bits, with scales and execution semantics specified separately.
QAT	Quantization-aware training: the learner forward path models a quantized representation; this need not reproduce native kernel arithmetic.
TIS / MIS	Truncated / masked importance sampling; bounded or discarded corrections alter the estimator.
AIS / CIS	Adaptive / calibrated importance sampling: estimator-mixture adaptation and confidence-dependent truncation, respectively.
ACR / UAQ	QuRL's Adaptive Clipping Range / Update-Aware Quantization; B2 update control and A2 update visibility.
TBPO / RAC	QaRL's Trust-Band Policy Optimization / QUADS's Residual Activation Compensation; distinct objective and numerical components.
PPO / GRPO	Proximal Policy Optimization / Group Relative Policy Optimization. Their loss roles require behavior and proximal policies to be distinguished.
ESS / P3O	Effective sample size / Policy-on Policy-off Policy Optimization; batch-ratio reliability controls update influence and regularization.
DPPO / DRPO	Divergence Proximal / Divergence Regularized Policy Optimization; divergence-based masking versus smooth corrective regularization.
MIPI / MIPU	Monotonic Inference Policy Improvement / Update; inference-policy objective and practical candidate construction/acceptance.
Rollout-ResQ	Sparse residual correction of rollout activation quantization; the reviewed HiFloat4 experiments simulate FP4.
TBIK	Tree-Based Invariant Kernels; matched reduction order across tensor-parallel sizes.
Partial rollout	Collection in which a response can span collection boundaries; it does not by itself imply concurrent generation and optimization.
Cache provenance	Parameter versions and numerical execution paths that produced retained cache entries.
Behavior / proximal / reference	Actual sampling law / update-ratio anchor / KL-regularization target; see Table 1.

References

[1] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, Jianfeng Gao. On the Rollout-Training Mismatch in Modern RL Systems. *OPT 2025: Optimization for Machine Learning*, 2025. https://www.opt-ml.org/papers/2025/paper116.pdf.

[2] Wei Huang, Yi Ge, Shuai Yang, Yicheng Xiao, Huizi Mao, Yujun Lin, Hanrong Ye, Sifei Liu, Ka Chun Cheung, Hongxu Yin, Yao Lu, Xiaojuan Qi, Song Han, Yukang Chen. QeRL: Beyond Efficiency - Quantization-enhanced Reinforcement Learning for LLMs. *International Conference on Learning Representations*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/a15a2ece7f0663d1ba7db91103ac61c9-Abstract-Conference.html.

[3] Haocheng Xi, Charlie Ruan, Peiyuan Liao, Yujun Lin, Han Cai, Yilong Zhao, Shuo Yang, Kurt Keutzer, Song Han, Ligeng Zhu. Jet-RL: Enabling On-Policy FP8 Reinforcement Learning with Unified Training and Rollout Precision Flow. *arXiv preprint*, 2026. https://arxiv.org/abs/2601.14243v2.

[4] Zhaopeng Qiu, Shuang Yu, Jingqi Zhang, Shuai Zhang, Xue Huang, Jingyi Yang, Junjie Lai. FP8-RL: A Practical and Stable Low-Precision Stack for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2601.18150v2.

[5] Yuhang Li, Reena Elangovan, Xin Dong, Priyadarshini Panda, Brucek Khailany. QuRL: Low-Precision Reinforcement Learning for Efficient Reasoning. *International Conference on Learning Representations*, 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/6b4880ced785077159b340710ceb7681-Abstract-Conference.html.

[6] Hao Gu, Hao Wang, Jiacheng Liu, Lujun Li, Qiyuan Zhu, Bei Liu, Binxing Xu, Lei Wang, Xintong Yang, Sida Lin, Sirui Han, Yike Guo. QaRL: Rollout-Aligned Quantization-Aware RL for Fast and Stable Training under Training–Inference Mismatch. *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. https://aclanthology.org/2026.findings-acl.841/.

[7] Jiajun Zhou, Wei Shao, Lingchao Zheng, Yuwei Fan, Ngai Wong. AIS: Adaptive Importance Sampling for Quantized RL. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.13907v1.

[8] Zhengyang Zhuge, Hao Yu, Xin Wang, Zheng Li, Yizhong Cao, Dayiheng Liu, Jianwei Zhang. QUADS: Stabilizing NVFP4 Reinforcement Learning for MoE via QUantization-error Alignment across Dual Sides. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.15810v1.

[9] Wenwu Fan, Qihong Lin, Zhijie Xia, Zhuo Zheng, Sihao Wang, Qiang Chen, Liangsheng Zhu. ACRL: Adaptive Control of Training-Inference Discrepancy for Stable Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.24062v1.

[10] Martin Marek, Max Ryabinin. Score Centering Stabilizes Off-policy Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.20807v1.

[11] Fanchao Chen, Ziheng Jiang, Ziyun Wei, Zheng Zhong, Du Li, Chi Zhang, Haibin Lin, Shivaram Venkataraman. Towards Full Pipeline FP8 Reinforcement Learning for LLMs. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.22870v1.

[12] Zhen Li, Shuai Zhang, Yanggan Gu, Yiming Zhang, Yang Yu, Mingfa Feng, Congkai Xie, Shuang Yu, Junjie Lai, Hongxia Yang. TRIAGE: Direction-Aware Mismatch Stabilization of Native NVFP4 Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.07043v2.

[13] Xin Wang, Hao Yu, Zhengyang Zhuge, Bochao Mao, Zheng Li, Junda Feng, Yuyan Luo, Yi Zhang, Yizhong Cao, Mi Zhang, Dayiheng Liu, Jianwei Zhang. TRACE: Rollout-Guided Quantization-Aware Training for FP4 Reinforcement Learning of MoE Language Models. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.07767v2.

[14] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, Junyang Lin. Stabilizing Reinforcement Learning with LLMs: Formulation and Practices. *arXiv preprint*, 2025. https://arxiv.org/abs/2512.01374v3.

[15] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, Fuli Luo. Stabilizing MoE Reinforcement Learning by Aligning Training and Inference Routers. *arXiv preprint*, 2025. https://arxiv.org/abs/2510.11370v2.

[16] Penghui Qi, Zichen Liu, Xiangxin Zhou, Tianyu Pang, Chao Du, Wee Sun Lee, Min Lin. Defeating the Training-Inference Mismatch via FP16. *arXiv preprint*, 2025. https://arxiv.org/abs/2510.26788v1.

[17] Sihwan Park, Sung-Yub Kim, Doohyuk Jang, Eunho Yang. KV4-RL: Unlocking Faster Asynchronous Rollouts for RLVR with INT4 KV Cache Quantization. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.14448v1.

[18] Yaxiang Zhang, Yingru Li, Jiacai Liu, Jiawei Xu, Ziniu Li, Qian Liu, Haoyuan Li. Beyond Precision: Training-Inference Mismatch is an Optimization Problem and Simple LR Scheduling Fixes It. *arXiv preprint*, 2026. https://arxiv.org/abs/2602.01826v1.

[19] Tianle Zhong, Neiwen Ling, Yifan Pi, Zijun Wei, Tianshu Yu, Geoffrey Fox, Peng Wu, Xiao Yu. Diagnosing Training Inference Mismatch in LLM Reinforcement Learning via a Zero-Mismatch Reference. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.14220v2.

[20] Tianrun Yu, Kaixiang Zhao, Shangzhe Li, Yuxiao Yang, Porter Jenkins, Weitong Zhang, Taylor W. Killian. Rethinking Training-Inference Mismatch in LLM Reinforcement Learning: Where It Arises and How to Correct It. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.32444v1.

[21] Rasool Fakoor, Murdock Aubry, Nicholas Stranges, Alexander J. Smola. Let the Batch Decide, On- or Off-Policy: Effective Sample Size for RL Post-Training. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.12380v2.

[22] Jiarui Yao, Xiangxin Zhou, Penghui Qi, Wee Sun Lee, Liefeng Bo, Tianyu Pang. Rethinking the Divergence Regularization in LLM RL. *arXiv preprint*, 2026. https://arxiv.org/abs/2606.09821v1.

[23] Jing Liang, Hongyao Tang, Yi Ma, Yancheng He, Weixun Wang, Xiaoyang Li, Ju Huang, Wenbo Su, Jinyi Liu, Yan Zheng, Jianye Hao, Bo Zheng. The Mirage of Optimizing Training Policies: Monotonic Inference Policies as the Real Objective for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2606.29526v1.

[24] Hei Yi Mak, Shadan Golestan, Hoang Le, Mehran Taghian Jazi, Yunke Peng, Yaoyuan Wang, Yao Wang, Junsong Wang, Tianchi Hu, Fengchen He, Guipeng Hu, Tanzila Rahman, Anandharaju Durai Raju. HiFloat4 Format for End-To-End Reinforcement Learning Post-Training of Large Language Models. *arXiv preprint*, 2026. https://arxiv.org/abs/2607.26515v1.

[25] RadixArk, Tom Chen, Mao Cheng, Shi Dong, Kangrui Du, Yanbin Jiang, Jiajun Li, Yiming Li, Tao Lin, Yusheng Su, Andy Ye, Yueming Yuan, Zhichen Zeng. Miles v0.1: Production-Level Post-Training. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.08368v1.

[26] Weili Xu, Jisen Li, Yuqing Jian, Chenxi Li, Zhizhou Sha, Yifan Yu, Qingyang Wu, Chenfeng Xu, Zhongzhu Zhou, Tianyi Zhang, Ben Athiwaratkun. QATFactory: A Versatile, Deployment-Aligned Framework for Quantization-aware Training and Distillation of LLMs. *arXiv preprint*, 2026. https://arxiv.org/abs/2609.39223v2.

[27] Shuo Han, Ziqing Hu. Baselines and Score Centering in Policy Gradient Estimation. *arXiv preprint*, 2026. https://arxiv.org/abs/2610.22263v1.

[28] Ziyang Zhang, Xinheng Ding, Jiayi Yuan, Rixin Liu, Huizi Mao, Jiarong Xing, Zirui Liu. Deterministic Inference across Tensor Parallel Sizes That Eliminates Training-Inference Mismatch. *arXiv preprint*, 2025. https://arxiv.org/abs/2511.17826v2.

[29] Penghui Qi, Xiangxin Zhou, Zichen Liu, Tianyu Pang, Chao Du, Min Lin, Wee Sun Lee. Rethinking the Trust Region in LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2602.04879v3.

[30] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint*, 2017. https://arxiv.org/abs/1707.06347.

[31] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint*, 2024. https://arxiv.org/abs/2402.03300.

[32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint*, 2023. https://arxiv.org/abs/2309.06180.

[33] Zhiwei Hao, Jianyuan Guo, Li Shen, Yong Luo, Han Hu, Guoxia Wang, Dianhai Yu, Yonggang Wen, Dacheng Tao. Low-Precision Training of Large Language Models: Methods, Challenges, and Opportunities. *arXiv preprint*, 2025. https://arxiv.org/abs/2505.01043.

[34] Rohan Surana, Gagan Mundada, Xunyi Jiang, Chuhan Wang, Zhenwei Tang, Difan Jiao, Zihan Huang, Yuxin Xiong, Junda Wu, Sheldon Yu, Xintong Li, Raghav Jain, Nikki Kuang, Sizhe Zhou, Bowen Jin, Zhendong Chu, Tong Yu, Ryan Rossi, Kuan-Hao Huang, Jingbo Shang, Jiawei Han, Julian McAuley. Generate, Filter, Control, Replay: A Comprehensive Survey of Rollout Strategies for LLM Reinforcement Learning. *arXiv preprint*, 2026. https://arxiv.org/abs/2605.02913.

[35] Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, Guoli Jia, Pengfei Li, Yu Fu, Xingtai Lv, Yuchen Zhang, Sihang Zeng, Shang Qu, Haozhan Li, Shijie Wang, Yuru Wang, Xinwei Long, Fangfu Liu, Xiang Xu, Jiaze Ma, Xuekai Zhu, Ermo Hua, Yihao Liu, Zonglin Li, Huayu Chen, Xiaoye Qu, Yafu Li, Weize Chen, Zhenzhao Yuan, Junqi Gao, Dong Li, Zhiyuan Ma, Ganqu Cui, Zhiyuan Liu, Biqing Qi, Ning Ding, Bowen Zhou. A Survey of Reinforcement Learning for Large Reasoning Models. *arXiv preprint*, 2025. https://arxiv.org/abs/2509.08827.